TUGAS BESAR 1
IF3070 DASAR INTELIGENSI ARTIFISIAL
PENCARIAN SOLUSI PENATAAN KENDARAAN PADA KAPAL KARGO DENGAN LOCAL SEARCH



Disusun oleh:

Syafaq Jiwa Raharjo	18224050
Faiz Muhammad Sauqi	18224086
Azfar Arafi Shofyan	18224096

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026

Daftar Isi

1. Deskripsi Persoalan **3**

2. Pembahasan **4**

2.1. Pemilihan Objective Function 4

2.2. Implementasi Algoritma Local Search 4

2.2.1. Algoritma Steepest Ascent Hill-Climbing 4

2.2.2. Algoritma Random Restart Hill-Climbing 8

2.2.3. Algoritma Sideways Hill-Climbing 11

2.2.4. Algoritma Stochastic Ascent Hill-Climbing 15

2.2.5. Algoritma Simulated Annealing 19

2.2.6. Algoritma Genetic Algorithm 34

2.3. Hasil Eksperimen dan Analisis 42

2.3.1. Algoritma Steepest Ascent Hill-Climbing 42

2.3.2. Algoritma Random Restart Hill-Climbing 46

2.3.3. Algoritma Sideways Hill-Climbing 51

2.3.4. Algoritma Stochastic Hill-Climbing 55

2.3.5. Algoritma Simulated Annealing 60

2.3.6. Algoritma Genetic Algorithm 66

2.3.6.1. Populasi Sebagai Kontrol 67

2.3.6.2. Iterasi Sebagai Kontrol 78

3. Kesimpulan dan Saran **91**

4. Pembagian Tugas **92**

5. Referensi **94**

SURAT PERNYATAAN PENGGUNAAN AI **95**

1. Deskripsi Persoalan

Sebuah perusahaan pelayaran mengoperasikan kapal kargo pengangkut kendaraan dan menerima kontrak pengiriman dari beberapa dealer sekaligus. Pada setiap jadwal keberangkatan, terdapat antrean kendaraan dengan ukuran, berat, dan tarif pengiriman yang berbeda-beda, sementara luas geladak dan kapasitas angkut kapal terbatas. Penataan yang selama ini dilakukan secara manual membuat ruang geladak sering terbuang dan keuntungan belum tentu maksimal, sehingga dibutuhkan cara untuk menentukan kendaraan yang diangkut beserta posisi dan orientasinya agar total tarif pengiriman (*ShippingFee*) sebesar mungkin.



Untuk itu, geladak dan kendaraan direpresentasikan sebagai grid dua dimensi. Setiap kendaraan memiliki atribut id, dimensi, orientasi, *ShippingFee*, berat, dan ETA, sedangkan kapal memiliki dimensi geladak dan kapasitas angkut maksimum (*MaxCapacity*). Solusi berupa susunan kendaraan di geladak, dengan kendaraan yang tidak terpilih berada di luar kapal. Susunan tersebut harus memenuhi tiga batasan: kendaraan tidak saling tumpang tindih, seluruh bagian kendaraan berada di dalam geladak, dan total berat tidak melebihi *MaxCapacity*.

Pada tugas besar ini, kami mengimplementasikan algoritma *local search*, yaitu Hill-Climbing, Simulated Annealing, dan Genetic Algorithm, untuk mencari susunan kendaraan dengan total *ShippingFee* sebesar mungkin. Melalui eksperimen, kami menganalisis efektivitas dan efisiensi masing-masing algoritma dalam menemukan solusi yang mendekati optimal berdasarkan nilai *objective function* akhir, durasi, dan banyak iterasi.

2. Pembahasan

2.1. Pemilihan Objective Function

Objective function merupakan ukuran yang digunakan untuk menilai seberapa baik suatu *state*. Pada persoalan penataan kendaraan di geladak kapal, aspek utama yang ditinjau adalah total *ShippingFee* dari kendaraan yang dimuat, sesuai dengan tujuan persoalan, yaitu memaksimalkan keuntungan pengiriman. Semakin besar total *ShippingFee*, semakin baik *state* tersebut. Namun, dua susunan yang berbeda dapat memiliki total *ShippingFee* yang sama. Oleh karena itu, kami menambahkan aspek kedua sebagai penentu (*tie-breaker*), yaitu luas ruang kosong berbentuk persegi panjang terbesar pada geladak. Susunan dengan ruang kosong yang lebih lega dinilai lebih baik, karena ruang tersebut masih dapat diisi kendaraan lain pada langkah berikutnya.

$$f(s) \;=\; TotalFee(s) \;+\; 0,99 \times L(s) \,/\, (w \times l)$$

TotalFee(s) : total *ShippingFee* kendaraan yang dimuat
L(s) : luas ruang kosong persegi panjang terbesar
w × l : luas geladak
0,99 : faktor agar *tie-breaker* < 1

Bagian *tie-breaker* dinormalisasi dengan luas geladak dan dikalikan 0,99 sehingga nilainya selalu berada di antara 0 dan 1. Karena *ShippingFee* bernilai bilangan bulat, *tie-breaker* tidak akan pernah mengalahkan selisih *ShippingFee* sekecil apa pun, sehingga hanya berperan ketika dua *state* memiliki total *ShippingFee* yang sama. Berbeda dengan pendekatan penalti, batasan persoalan (tidak tumpang tindih, berada di dalam geladak, dan total berat tidak melebihi *MaxCapacity*) tidak dimasukkan ke dalam *objective function*. *State* yang melanggar batasan langsung ditolak saat pembangkitan *neighbor*, sehingga setiap *state* yang dinilai oleh algoritma dijamin valid. Nilai *ShippingFee* yang dilaporkan sebagai hasil akhir adalah TotalFee(s) tanpa *tie-breaker*.

2.2. Implementasi Algoritma Local Search

2.2.1. Algoritma *Steepest Ascent Hill-Climbing*

Steepest Ascent Hill-Climbing merupakan algoritma yang pada setiap iterasi mengevaluasi seluruh *neighbor* dari *state* saat ini, lalu berpindah ke *neighbor* dengan nilai *objective* tertinggi. Pada persoalan ini, *state* adalah satu susunan kendaraan di geladak kapal, sedangkan *neighbor* adalah susunan lain yang diperoleh dari satu

perubahan kecil, seperti menggeser, memutar, menukar, mengeluarkan, atau memasukkan kendaraan. Pencarian berhenti ketika tidak ada lagi *neighbor* yang dapat menaikkan total *ShippingFee*, dan susunan terakhir tersebut menjadi rekomendasi penataan muatan kapal.

Berikut merupakan *source code* beserta penjelasan dari algoritma *Steepest Ascent Hill-Climbing*.

```
import time

from src.initial import random_initial_state
from src.objective import objective
from src.neighbors import get_all_neighbors,
get_random_neighbor



def hill_climb(problem, rng, initial_state=None):
   start_time = time.perf_counter()
   used_fallback = False
   if initial_state is None:
       initial_state, used_fallback =
random_initial_state(problem, rng)

   current = initial_state
   current_value = objective(current)
   history = [current_value]
   moves = []

   while True:
       best_move, best_state, best_value = None, None,
current_value
       for move, neighbor in get_all_neighbors(current):
           value = objective(neighbor)
           if value > best_value:
               best_move, best_state, best_value = move,
```

```
neighbor, value
        if best_state is None:
            break
        current, current_value = best_state, best_value
        history.append(current_value)
        moves.append(best_move)

    return {
        "algorithm": "hc",
        "initial_state": initial_state,
        "final_state": current,
        "history": history,
        "moves": moves,
        "iterations": len(moves),
        "duration_s": time.perf_counter() - start_time,
        "used_fallback": used_fallback,
    }
```

Nama Fungsi	`hill_climb(problem, rng, initial_state=None)`
Deskripsi	Menjalankan alur *Steepest Ascent Hill-Climbing*. Dimulai dari *state* awal (dibangkitkan acak jika tidak diberikan), lalu pada setiap iterasi seluruh *neighbor* valid dievaluasi dan dipilih yang nilai *objective*-nya tertinggi. *State* berpindah jika nilainya lebih baik, dan pencarian berhenti ketika tidak ada *neighbor* yang lebih baik.
Output	*Dictionary* berisi *state* awal dan akhir, riwayat nilai *objective*, daftar *move*, banyak iterasi, durasi, dan status *fallback*.

Source Code	`def hill_climb(problem, rng,`
	`initial_state=None):`
	`  start_time = time.perf_counter()`
	`  used_fallback = False`
	`  if initial_state is None:`
	`  initial_state, used_fallback =`

```
random_initial_state(problem, rng)


   current = initial_state
   current_value = objective(current)
   history = [current_value]
   moves = []

   while True:
       best_move, best_state, best_value = None,
None, current_value
       for move, neighbor in
get_all_neighbors(current):
           value = objective(neighbor)
           if value > best_value:
               best_move, best_state, best_value
= move, neighbor, value
       if best_state is None:
           break
       current, current_value = best_state,
best_value
       history.append(current_value)
       moves.append(best_move)

   return {
       "algorithm": "hc",
       "initial_state": initial_state,
       "final_state": current,
       "history": history,
       "moves": moves,
       "iterations": len(moves),
       "duration_s": time.perf_counter() -
start_time,
       "used_fallback": used_fallback,
   }
```

Nama Fungsi	`get_all_neighbors(state)`
Deskripsi	Membangkitkan seluruh *neighbor* dari susunan saat ini dengan lima jenis *move*: menukar, menggeser, memutar, mengeluarkan, dan memasukkan kendaraan. *Neighbor* yang membuat kendaraan bertumpuk, keluar geladak, atau melebihi kapasitas langsung dibuang.
Output	*List* pasangan (*move*, *state neighbor*) yang valid.

Source Code	`def get_all_neighbors(state):`
	`  result = []`
	`  for move in all_moves(state):`
	`  new = apply_move(state, move)`
	`  if is_valid(new):`
	`  result.append((move, new))`
	`  return result`

Nama Fungsi	`objective(state)`
Deskripsi	Menghitung nilai susunan, yaitu total *ShippingFee* kendaraan yang dimuat ditambah *tie-breaker* 0,99 × luas persegi panjang kosong terbesar / (w × l).
Output	Nilai *objective* berupa bilangan real.
Source Code	`def objective(state):` `  return total_fee(state) + tie_breaker(state)`

2.2.2. Algoritma *Random Restart Hill-Climbing*

Random Restart Hill-Climbing merupakan pengembangan dari *Steepest Ascent Hill-Climbing* untuk mengatasi pencarian yang terjebak di *local maximum*. Algoritma ini menjalankan *Steepest Ascent* berulang kali, dan setiap pengulangan (*restart*) dimulai dari susunan kendaraan acak yang baru. Dari semua pengulangan, susunan dengan nilai *objective* tertinggi dipilih sebagai rekomendasi penataan muatan. Dengan begitu,

peluang menemukan susunan dengan total *ShippingFee* yang lebih besar ikut meningkat.

Berikut merupakan *source code* beserta penjelasan dari algoritma *Random Restart Hill-Climbing*.

```
import time

from src.initial import random_initial_state
from src.objective import objective
from src.neighbors import get_all_neighbors,
get_random_neighbor

def hill_climb_random_restart(problem, rng,
initial_state=None, max_restarts=10):
   start_time = time.perf_counter()
   first = hill_climb(problem, rng, initial_state)
   best = first
   history = list(first["history"])
   iterations_per_restart = [first["iterations"]]
   used_fallback = first["used_fallback"]

   for _ in range(max_restarts):
       result = hill_climb(problem, rng)
       history += result["history"]
       iterations_per_restart.append(result["iterations"])
       used_fallback = used_fallback or
result["used_fallback"]
       if result["history"][-1] > best["history"][-1]:
           best = result

   return {
       "algorithm": "hc_random_restart",
       "params": {"max_restarts": max_restarts},
       "initial_state": first["initial_state"],
```

```
        "final_state": best["final_state"],
        "history": history,
        "moves": best["moves"],
        "iterations": sum(iterations_per_restart),
        "restarts": max_restarts,
        "iterations_per_restart": iterations_per_restart,
        "duration_s": time.perf_counter() - start_time,
        "used_fallback": used_fallback,
    }
```

Nama Fungsi	`hill_climb_random_restart(problem, rng, initial_state=None, max_restarts=10)`
Deskripsi	Menjalankan hill_climb sekali dari *state* awal, lalu mengulanginya sebanyak max_restarts kali dari *state* acak baru. Setiap kali, hasilnya dibandingkan dan disimpan yang nilai *objective* akhirnya tertinggi. Riwayat *objective* semua pengulangan disambung menjadi satu, dan banyak iterasi tiap pengulangan dicatat. Pencarian berhenti setelah banyak *restart* mencapai max_restarts.
Output	*Dictionary* berisi *state* awal, *state* terbaik dari semua pengulangan, riwayat *objective* gabungan, parameter max_restarts, banyak *restart*, banyak iterasi per *restart*, total iterasi, durasi, dan status *fallback*.

Source Code	`def hill_climb_random_restart(problem, rng, initial_state=None, max_restarts=10):`
	`  start_time = time.perf_counter()`
	`  first = hill_climb(problem, rng, initial_state)`
	`  best = first`
	`  history = list(first["history"])`
	`  iterations_per_restart = [first["iterations"]]`
	`  used_fallback = first["used_fallback"]`

```
   for _ in range(max_restarts):
       result = hill_climb(problem, rng)
       history += result["history"]

iterations_per_restart.append(result["iterations
"])
       used_fallback = used_fallback or
result["used_fallback"]
       if result["history"][-1] >
best["history"][-1]:
           best = result

   return {
       "algorithm": "hc_random_restart",
       "params": {"max_restarts": max_restarts},
       "initial_state": first["initial_state"],
       "final_state": best["final_state"],
       "history": history,
       "moves": best["moves"],
       "iterations":
sum(iterations_per_restart),
       "restarts": max_restarts,
       "iterations_per_restart":
iterations_per_restart,
       "duration_s": time.perf_counter() -
start_time,
       "used_fallback": used_fallback,
   }
```

2.2.3. Algoritma *Sideways Hill-Climbing*

Hill-Climbing with Sideways Move merupakan pengembangan dari *Steepest Ascent Hill-Climbing* yang mengizinkan perpindahan ke *neighbor* dengan nilai *objective* yang

sama (*sideways move*). Pada persoalan penataan muatan, sering ada beberapa susunan kendaraan dengan total *ShippingFee* dan luas ruang kosong yang sama, sehingga pencarian dapat berhenti di dataran (*shoulder* atau *flat local maximum*). Dengan *sideways move*, pencarian dapat terus berpindah di dataran tersebut untuk mencari jalan menuju susunan yang lebih baik. Agar tidak berputar tanpa akhir, banyak *sideways move* dibatasi oleh parameter *maximum sideways move*.

Berikut merupakan *source code* beserta penjelasan dari algoritma *Hill-Climbing with Sideways Move*.

.

```
import time

from src.initial import random_initial_state
from src.objective import objective
from src.neighbors import get_all_neighbors,
get_random_neighbor

def hill_climb_sideways(problem, rng, initial_state=None,
max_sideways=100):
    start_time = time.perf_counter()
    used_fallback = False
    if initial_state is None:
        initial_state, used_fallback =
random_initial_state(problem, rng)

    current = initial_state
    current_value = objective(current)
    history = [current_value]
    moves = []
    sideways = 0

    while True:
        best_value = None
        best = []
```

```
        for move, neighbor in get_all_neighbors(current):
            value = objective(neighbor)
            if best_value is None or value > best_value:
                best_value, best = value, [(move, neighbor)]
            elif value == best_value:
                best.append((move, neighbor))
        if best_value is None or best_value < current_value:
            break
        if best_value == current_value:
            if sideways >= max_sideways:
                break
            sideways += 1
        move, current = rng.choice(best)
        current_value = best_value
        history.append(current_value)
        moves.append(move)

    return {
        "algorithm": "hc_sideways",
        "params": {"max_sideways": max_sideways},
        "initial_state": initial_state,
        "final_state": current,
        "history": history,
        "moves": moves,
        "iterations": len(moves),
        "sideways_moves": sideways,
        "duration_s": time.perf_counter() - start_time,
        "used_fallback": used_fallback,
    }
```

Nama Fungsi	`hill_climb_sideways(problem, rng, initial_state=None, max_sideways=100)`

Deskripsi	Pada setiap iterasi, seluruh *neighbor* valid dievaluasi dan dikumpulkan *neighbor* dengan nilai *objective* tertinggi. Jika nilainya lebih tinggi dari *state* saat ini, *state* berpindah seperti biasa. Jika nilainya sama, perpindahan dihitung sebagai *sideways move* dan tetap dilakukan selama jumlahnya belum mencapai max_sideways. Jika ada beberapa *neighbor* terbaik yang seri, salah satunya dipilih secara acak supaya pencarian tidak bolak-balik di dua susunan yang sama. Pencarian berhenti ketika *neighbor* terbaik lebih buruk dari *state* saat ini atau batas *sideways move* sudah tercapai.
Output	*Dictionary* berisi *state* awal dan akhir, riwayat nilai *objective*, daftar *move*, parameter max_sideways, banyak *sideways move* yang terpakai, banyak iterasi, durasi, dan status *fallback*.

Source Code	`def hill_climb_sideways(problem, rng, initial_state=None, max_sideways=100):`
	`  start_time = time.perf_counter()`
	`  used_fallback = False`
	`  if initial_state is None:`
	`  initial_state, used_fallback = random_initial_state(problem, rng)`
	
	`  current = initial_state`
	`  current_value = objective(current)`
	`  history = [current_value]`
	`  moves = []`
	`  sideways = 0`
	
	`  while True:`
	`  best_value = None`
	`  best = []`
	`  for move, neighbor in get_all_neighbors(current):`
	`  value = objective(neighbor)`
	`  if best_value is None or value > best_value:`

```
                best_value, best = value, [(move,
neighbor)]
            elif value == best_value:
                best.append((move, neighbor))
        if best_value is None or best_value <
current_value:
            break
        if best_value == current_value:
            if sideways >= max_sideways:
                break
            sideways += 1
        move, current = rng.choice(best)
        current_value = best_value
        history.append(current_value)
        moves.append(move)

    return {
        "algorithm": "hc_sideways",
        "params": {"max_sideways": max_sideways},
        "initial_state": initial_state,
        "final_state": current,
        "history": history,
        "moves": moves,
        "iterations": len(moves),
        "sideways_moves": sideways,
        "duration_s": time.perf_counter() -
start_time,
        "used_fallback": used_fallback,
    }
```

2.2.4. Algoritma *Stochastic Ascent Hill-Climbing*

Stochastic Hill-Climbing merupakan varian hill-climbing yang tidak mengevaluasi seluruh neighbor. Pada setiap iterasi, algoritma hanya membangkitkan satu neighbor

secara acak, lalu berpindah ke neighbor tersebut jika nilai objective-nya lebih tinggi dari state saat ini. Pada persoalan penataan muatan, banyaknya kemungkinan perubahan susunan (terutama memasukkan kendaraan ke sel mana pun di geladak) membuat evaluasi seluruh neighbor cukup mahal, sehingga cara ini membuat setiap iterasi jauh lebih ringan. Pencarian berhenti setelah banyak iterasi mencapai batas maksimum yang ditentukan.

Berikut merupakan source code beserta penjelasan dari algoritma Stochastic Hill-Climbing.

```
import time

from src.initial import random_initial_state
from src.objective import objective
from src.neighbors import get_all_neighbors,
get_random_neighbor

def hill_climb_stochastic(problem, rng, initial_state=None,
max_iterations=1000):
    start_time = time.perf_counter()
    used_fallback = False
    if initial_state is None:
        initial_state, used_fallback =
random_initial_state(problem, rng)

    current = initial_state
    current_value = objective(current)
    history = [current_value]
    moves = []
    invalid = 0

    for _ in range(max_iterations):
        move, neighbor = get_random_neighbor(current, rng)
        if neighbor is None:
            invalid += 1
```

```
        else:
            value = objective(neighbor)
            if value > current_value:
                current, current_value = neighbor, value
                moves.append(move)
        history.append(current_value)

    return {
        "algorithm": "hc_stochastic",
        "params": {"max_iterations": max_iterations},
        "initial_state": initial_state,
        "final_state": current,
        "history": history,
        "moves": moves,
        "iterations": max_iterations,
        "invalid_neighbors": invalid,
        "duration_s": time.perf_counter() - start_time,
        "used_fallback": used_fallback,
    }
```

Nama Fungsi	`hill_climb_stochastic(problem, rng, initial_state=None, max_iterations=1000)`
Deskripsi	Pada setiap iterasi, satu *neighbor* acak dibangkitkan dengan get_random_neighbor. Jika *neighbor* tidak valid, iterasi tersebut dicatat sebagai *neighbor* tidak valid. Jika valid dan nilai *objective*-nya lebih tinggi, *state* berpindah ke *neighbor* tersebut. Jika tidak, *state* tetap. Nilai *objective state* saat ini dicatat di setiap iterasi. Pencarian berhenti setelah max_iterations iterasi.
Output	*Dictionary* berisi *state* awal dan akhir, riwayat nilai *objective* per iterasi, daftar *move* yang diambil, parameter max_iterations, banyak *neighbor* tidak valid, banyak iterasi, durasi, dan status *fallback*.

Source Code	hill_climb_stochastic(problem, rng, initial_state=None, max_iterations=1000):
	   start_time = time.perf_counter()
	   used_fallback = False
	   if initial_state is None:
	       initial_state, used_fallback = random_initial_state(problem, rng)
	
	   current = initial_state
	   current_value = objective(current)
	   history = [current_value]
	   moves = []
	   invalid = 0
	
	   for _ in range(max_iterations):
	       move, neighbor = get_random_neighbor(current, rng)
	       if neighbor is None:
	           invalid += 1
	       else:
	           value = objective(neighbor)
	           if value > current_value:
	               current, current_value = neighbor, value
	               moves.append(move)
	       history.append(current_value)
	
	   return {
	       "algorithm": "hc_stochastic",
	       "params": {"max_iterations": max_iterations},
	       "initial_state": initial_state,
	       "final_state": current,
	       "history": history,

	"moves": moves, "iterations": max_iterations, "invalid_neighbors": invalid, "duration_s": time.perf_counter() - start_time, "used_fallback": used_fallback, }

Nama Fungsi	get_random_neighbor(state, rng)
Deskripsi	Memilih satu jenis *move* secara acak di antara jenis yang mungkin pada *state* saat ini (menukar, menggeser, memutar, mengeluarkan, atau memasukkan kendaraan), lalu menerapkannya. Hasilnya diperiksa terhadap seluruh *constraint*.
Output	Pasangan (*move*, *state neighbor*), atau (*move*, None) jika *neighbor* melanggar *constraint*.
Source Code	get_random_neighbor(state, rng): move = random_move(state, rng) new = apply_move(state, move) if is_valid(new): return move, new return move, None

2.2.5. Algoritma Simulated Annealing

Simulated Annealing merupakan algoritma local search yang menggunakan pendekatan probabilistik untuk mencari solusi dengan nilai *objective function* setinggi mungkin. Berbeda dengan Hill-Climbing yang pada umumnya hanya menerima perpindahan menuju solusi yang lebih baik, Simulated Annealing memungkinkan perpindahan ke solusi yang lebih buruk dengan probabilitas tertentu. Mekanisme ini digunakan untuk meningkatkan kemampuan eksplorasi dan mengurangi risiko pencarian berhenti pada *local optimum*.

Pada persoalan penataan kendaraan di geladak kapal, algoritma Simulated Annealing

memulai pencarian dari state awal yang dibangkitkan secara acak. Pada setiap iterasi, algoritma membangkitkan satu neighbor melalui fungsi get_random_neighbor() dengan operasi swap, shift, rotate, remove, atau insert. Neighbor yang melanggar constraint ditolak, sedangkan neighbor yang valid dievaluasi menggunakan objective function yang telah ditentukan.

Algoritma menghitung perubahan nilai objective menggunakan rumus:

$\Delta E = f(\text{neighbor}) - f(\text{current})$

Apabila $\Delta E \geq 0$, neighbor langsung diterima. Sebaliknya, apabila $\Delta E < 0$, neighbor dapat diterima berdasarkan probabilitas:

$P = e^{(\Delta E/T)}$

dengan T merupakan temperatur pada iterasi tersebut. Temperatur diturunkan menggunakan geometric cooling schedule:

$T(t) = T_0 \times \alpha^t$

Pada implementasi ini, digunakan nilai bawaan $T_0 = 60$, $\alpha = 0{,}995$, dan maksimum iterasi sebanyak 1.000. Dengan penurunan temperatur, kemungkinan menerima neighbor yang lebih buruk semakin kecil untuk nilai ΔE negatif yang sama.

Selama proses pencarian, algoritma menyimpan current state dan best state secara terpisah. Best state merupakan konfigurasi dengan nilai objective tertinggi yang pernah ditemukan sehingga solusi terbaik tetap tersimpan meskipun algoritma menerima perpindahan yang lebih buruk. Selain itu, dilakukan pencatatan riwayat objective, temperatur, probabilitas penerimaan, jumlah neighbor tidak valid, dan jumlah accepted worse moves.

Untuk menganalisis kondisi local optimum, implementasi juga menyediakan fungsi diagnosis yang memeriksa keberadaan neighbor dengan objective lebih tinggi serta menghitung frekuensi episode ketika pencarian memasuki local maximum. Diagnosis dilakukan secara terpisah dari pencarian utama.

Berikut merupakan source code beserta penjelasan fungsi-fungsi yang digunakan dalam implementasi algoritma Simulated Annealing.

```
import time
import math

from src.initial import random_initial_state
from src.objective import objective
from src.neighbors import get_all_neighbors,
get_random_neighbor

def cooling_schedule(t, initial_temp, cooling_rate,
max_iterations):
    if not isinstance(t, int) or t < 0:
        raise ValueError("Indeks iterasi harus bilangan bulat
non-negatif")

    if t >= max_iterations:
        return 0.0

    return initial_temp * (cooling_rate ** t)


def is_local_maximum(state):

    current_value = objective(state)

    for move, neighbor in get_all_neighbors(state):
        neighbor_value = objective(neighbor)

        if neighbor_value > current_value:
            return False

    return True
```

```
def count_stuck_episodes(visited_states):

    stuck_count = 0
    was_local_maximum = False

    for state in visited_states:
        current_is_local_maximum = is_local_maximum(state)

        if current_is_local_maximum and not was_local_maximum:
            stuck_count += 1

        was_local_maximum = current_is_local_maximum

    return stuck_count

def simulated_annealing(
    problem,
    rng,
    initial_state=None,
    initial_temp=60.0,
    cooling_rate=0.995,
    max_iterations=1000,
):

    if initial_temp <= 0:
        raise ValueError("initial_temp harus lebih dari 0")

    if not 0 < cooling_rate < 1:
        raise ValueError("cooling_rate harus di antara 0 dan
1")

    if not isinstance(max_iterations, int) or max_iterations
<= 0:
        raise ValueError("max_iterations harus bilangan bulat
```

```
positif")

    start_time = time.perf_counter()
    used_fallback = False

    if initial_state is None:
        initial_state, used_fallback =
random_initial_state(problem, rng)



    current = initial_state
    current_value = objective(current)

    best = current
    best_value = current_value

    history = [current_value]
    best_history = [best_value]
    moves = []
    iterations = 0
    visited_states = [current]
    temperature_history = []
    delta_history = []
    acceptance_probability_history = []
    accepted_history = []

    invalid_neighbors = 0
    accepted_worse_moves = 0



    for t in range(max_iterations):
        temperature = cooling_schedule(
            t,
            initial_temp,
```

```
            cooling_rate,
            max_iterations,
        )

        if temperature <= 0:
            break



        temperature_history.append(temperature)

        move, neighbor = get_random_neighbor(current, rng)

        delta_e = None
        acceptance_probability = None
        accepted = False

        if neighbor is None:
            invalid_neighbors += 1

        else:
            neighbor_value = objective(neighbor)
            delta_e = neighbor_value - current_value

            if delta_e >= 0:
                acceptance_probability = 1.0
                accepted = True
            else:
                acceptance_probability = math.exp(
                    delta_e / temperature
                )
                accepted = (
                    rng.random() < acceptance_probability
                )
```

```
            if accepted:
                if delta_e < 0:
                    accepted_worse_moves += 1

                current = neighbor
                current_value = neighbor_value
                moves.append(move)
                visited_states.append(current)

                if current_value > best_value:
                    best = current
                    best_value = current_value

        delta_history.append(delta_e)
        acceptance_probability_history.append(
            acceptance_probability
        )
        accepted_history.append(accepted)

        iterations += 1
        history.append(current_value)
        best_history.append(best_value)

    duration_s = time.perf_counter() - start_time


    diagnosis_start = time.perf_counter()

    stuck_count = count_stuck_episodes(visited_states)

    diagnosis_duration_s = (
        time.perf_counter() - diagnosis_start
    )
```

```
    return {
        "algorithm": "sa",
        "params": {
            "initial_temp": initial_temp,
            "cooling_rate": cooling_rate,
            "max_iterations": max_iterations,
        },
        "initial_state": initial_state,
        "final_state": current,
        "best_state": best,
        "history": history,
        "best_history": best_history,
        "moves": moves,
        "iterations": iterations,
        "duration_s": duration_s,
        "used_fallback": used_fallback,
        "temperature_history": temperature_history,
        "delta_history": delta_history,
        "acceptance_probability_history": (
            acceptance_probability_history
        ),
        "accepted_history": accepted_history,
        "invalid_neighbors": invalid_neighbors,
        "invalid_neighbor_percentage": (
            100.0 * invalid_neighbors / iterations
            if iterations > 0 else 0.0
        ),
        "accepted_worse_moves": accepted_worse_moves,
        "stuck_count": stuck_count,
        "diagnosis_duration_s": diagnosis_duration_s,
    }
```

Nama Fungsi	`simulated_annealing(problem, rng, initial_state=None, initial_temp=60.0, cooling_rate=0.995, max_iterations=1000)`
Deskripsi	Menjalankan proses Simulated Annealing dengan membangkitkan satu neighbor acak pada setiap iterasi. Neighbor yang lebih baik atau sama diterima, sedangkan neighbor yang lebih buruk dapat diterima berdasarkan probabilitas eksponensial dan temperatur saat itu. Algoritma menyimpan current state serta best state, mencatat riwayat pencarian, dan berhenti setelah mencapai batas maksimum iterasi.
Output	Dictionary berisi state awal, akhir, dan terbaik, riwayat objective, parameter algoritma, temperatur, perubahan objective, probabilitas penerimaan, jumlah invalid neighbor, accepted worse moves, stuck episodes, iterasi, serta durasi pencarian dan diagnosis.

Source Code	`def simulated_annealing(`
	`  problem,`
	`  rng,`
	`  initial_state=None,`
	`  initial_temp=60.0,`
	`  cooling_rate=0.995,`
	`  max_iterations=1000,`
	`):`
	
	`  if initial_temp <= 0:`
	`  raise ValueError("initial_temp harus lebih dari 0")`
	
	`  if not 0 < cooling_rate < 1:`
	`  raise ValueError("cooling_rate harus di antara 0 dan 1")`
	
	`  if not isinstance(max_iterations, int) or max_iterations <= 0:`
	`  raise ValueError("max_iterations harus`

```
bilangan bulat positif")

    start_time = time.perf_counter()
    used_fallback = False

    if initial_state is None:
        initial_state, used_fallback =
random_initial_state(problem, rng)



    current = initial_state
    current_value = objective(current)

    best = current
    best_value = current_value

    history = [current_value]
    best_history = [best_value]
    moves = []
    iterations = 0
    visited_states = [current]
    temperature_history = []
    delta_history = []
    acceptance_probability_history = []
    accepted_history = []

    invalid_neighbors = 0
    accepted_worse_moves = 0



    for t in range(max_iterations):
        temperature = cooling_schedule(
            t,
            initial_temp,
```

```
            cooling_rate,
            max_iterations,
        )

        if temperature <= 0:
            break



        temperature_history.append(temperature)

        move, neighbor =
get_random_neighbor(current, rng)



        delta_e = None
        acceptance_probability = None
        accepted = False

        if neighbor is None:
            invalid_neighbors += 1

        else:
            neighbor_value = objective(neighbor)
            delta_e = neighbor_value -
current_value

            if delta_e >= 0:
                acceptance_probability = 1.0
                accepted = True
            else:
                acceptance_probability =
math.exp(
                    delta_e / temperature
```

```
                )
                accepted = (
                    rng.random() <
acceptance_probability
                )

            if accepted:
                if delta_e < 0:
                    accepted_worse_moves += 1

                current = neighbor
                current_value = neighbor_value
                moves.append(move)
                visited_states.append(current)

                if current_value > best_value:
                    best = current
                    best_value = current_value

        delta_history.append(delta_e)
        acceptance_probability_history.append(
            acceptance_probability
        )
        accepted_history.append(accepted)

        iterations += 1
        history.append(current_value)
        best_history.append(best_value)


    duration_s = time.perf_counter() -
start_time
```

```
    diagnosis_start = time.perf_counter()

    stuck_count =
count_stuck_episodes(visited_states)

    diagnosis_duration_s = (
        time.perf_counter() - diagnosis_start
    )



    return {
        "algorithm": "sa",
        "params": {
            "initial_temp": initial_temp,
            "cooling_rate": cooling_rate,
            "max_iterations": max_iterations,
        },
        "initial_state": initial_state,
        "final_state": current,
        "best_state": best,
        "history": history,
        "best_history": best_history,
        "moves": moves,
        "iterations": iterations,
        "duration_s": duration_s,
        "used_fallback": used_fallback,
        "temperature_history":
temperature_history,
        "delta_history": delta_history,
        "acceptance_probability_history": (
            acceptance_probability_history
        ),
        "accepted_history": accepted_history,
        "invalid_neighbors": invalid_neighbors,
```

	"invalid_neighbor_percentage": (100.0 * invalid_neighbors / iterations if iterations > 0 else 0.0), "accepted_worse_moves": accepted_worse_moves, "stuck_count": stuck_count, "diagnosis_duration_s": diagnosis_duration_s, }

Nama Fungsi	cooling_schedule(t, initial_temp, cooling_rate, max_iterations)
Deskripsi	Menghitung temperatur pada iterasi ke-t menggunakan geometric cooling schedule, yaitu T(t) = $T_0 \times \alpha^t$. Temperatur terus menurun selama iterasi pencarian dan bernilai nol ketika batas iterasi telah tercapai.
Output	Nilai temperatur berupa bilangan real.

Source Code	def cooling_schedule(t, initial_temp, cooling_rate, max_iterations):
	
	if not isinstance(t, int) or t < 0:
	raise ValueError("Indeks iterasi harus bilangan bulat non-negatif")
	
	if t >= max_iterations:
	return 0.0

	`return initial_temp * (cooling_rate ** t)`

Nama Fungsi	`is_local_maximum(state)`
Deskripsi	Memeriksa apakah suatu state merupakan local maximum dengan membandingkan nilai objective state tersebut terhadap seluruh neighbor valid. State dikategorikan sebagai local maximum apabila tidak ditemukan neighbor valid dengan nilai objective yang lebih tinggi.
Output	Nilai Boolean yang menunjukkan apakah state merupakan local maximum.

Source Code	`def is_local_maximum(state):`
	`  current_value = objective(state)`
	
	`  for move, neighbor in get_all_neighbors(state):`
	`  neighbor_value = objective(neighbor)`
	
	`  if neighbor_value > current_value:`
	`  return False`
	
	`  return True`

Nama Fungsi	`count_stuck_episodes(visited_states)`
Deskripsi	Menghitung banyaknya episode ketika lintasan state yang diterima memasuki kondisi local maximum. Perhitungan dilakukan berdasarkan perubahan status dari bukan local maximum menjadi local maximum, sehingga tidak menghitung setiap iterasi dalam kondisi tersebut sebagai episode baru.

Output	Bilangan bulat berupa jumlah episode local maximum yang terdeteksi.

Source Code	`def count_stuck_episodes(visited_states):`
	`  stuck_count = 0`
	`  was_local_maximum = False`
	
	`  for state in visited_states:`
	`  current_is_local_maximum = is_local_maximum(state)`
	
	`  if current_is_local_maximum and not was_local_maximum:`
	`  stuck_count += 1`
	
	`  was_local_maximum = current_is_local_maximum`
	
	`  return stuck_count`

2.2.6. Algoritma *Genetic Algorithm*

Genetic Algorithm merupakan algoritma *local search* yang bekerja dengan sekumpulan *state* sekaligus yang disebut populasi. Pada persoalan ini, setiap individu adalah satu susunan kendaraan di geladak kapal, dan nilai *fitness*-nya dihitung dengan fungsi *objective* yang sama dengan algoritma lain. Pada setiap iterasi, dua induk dipilih dengan seleksi *roulette wheel*, digabungkan melalui *crossover* menjadi anak, lalu anak tersebut dapat mengalami mutasi. Individu terbaik selalu dipertahankan ke generasi berikutnya (elitisme), sedangkan kendaraan yang membuat anak melanggar *constraint* ditempatkan di luar kapal sehingga seluruh individu tetap valid. Pencarian berhenti setelah banyak iterasi mencapai batas yang ditentukan.

Berikut merupakan *source code* beserta penjelasan dari algoritma *Genetic Algorithm*.

```
import time
```

```
from src.constraints import is_valid
from src.initial import random_initial_state
from src.models import State
from src.neighbors import get_random_neighbor
from src.objective import objective

def make_population(problem, size, rng, p):
    population = []
    used_fallback = False
    for _ in range(size):
        state, fallback = random_initial_state(problem, rng,
p)
        population.append(state)
        used_fallback = used_fallback or fallback
    return population, used_fallback



def roulette(population, scores, rng):
    total = sum(scores)
    if total <= 0:
        return rng.choice(population)
    target = rng.uniform(0, total)
    acc = 0.0
    for individual, score in zip(population, scores):
        acc += score
        if acc >= target:
            return individual
    return population[-1]

def crossover(a, b, rng):
    child = State.empty(a.problem)
    order = list(range(len(a.problem.vehicles)))
```

```
    rng.shuffle(order)
    for i in order:
        parents = [a, b] if rng.random() < 0.5 else [b, a]
        for parent in parents:
            if not parent.is_inside(i):
                continue
            child.positions[i] = parent.positions[i]
            child.orientations[i] = parent.orientations[i]
            if is_valid(child):
                break
            child.positions[i] = None
            child.orientations[i] = None
    return child

def mutate(state, rate, rng):
    if rng.random() >= rate:
        return state
    _, neighbor = get_random_neighbor(state, rng)
    return neighbor if neighbor is not None else state

def best_index(scores):
    return max(range(len(scores)), key=scores.__getitem__)

def genetic_algorithm(
    problem,
    rng,
    population_size=50,
    iterations=100,
    p=0.3,
    crossover_rate=0.9,
    mutation_rate=0.3,
    elite=1,
):
```

```
    start_time = time.perf_counter()

    population, used_fallback = make_population(problem,
population_size, rng, p)
    scores = [objective(s) for s in population]

    initial_state = population[best_index(scores)]
    history = [max(scores)]
    history_avg = [sum(scores) / population_size]

    for _ in range(iterations):
        ranked = sorted(range(population_size),
key=scores.__getitem__, reverse=True)
        next_population = [population[k] for k in
ranked[:elite]]
        next_scores = [scores[k] for k in ranked[:elite]]
        while len(next_population) < population_size:
            a = roulette(population, scores, rng)
            b = roulette(population, scores, rng)
            if rng.random() < crossover_rate:
                child = crossover(a, b, rng)
            else:
                child = a.copy()
            child = mutate(child, mutation_rate, rng)
            next_population.append(child)
            next_scores.append(objective(child))
        population, scores = next_population, next_scores
        history.append(max(scores))
        history_avg.append(sum(scores) / population_size)

    return {
        "algorithm": "ga",
        "initial_state": initial_state,
```

```
        "final_state": population[best_index(scores)],
        "history": history,
        "history_avg": history_avg,
        "population_size": population_size,
        "iterations": iterations,
        "duration_s": time.perf_counter() - start_time,
        "used_fallback": used_fallback,
    }
```

Nama Fungsi	`genetic_algorithm(problem, rng, population_size=50, iterations=100, p=0.3, crossover_rate=0.9, mutation_rate=0.3, elite=1)`
Deskripsi	Menjalankan alur *Genetic Algorithm*. Populasi awal dibangkitkan secara acak, lalu pada setiap iterasi individu terbaik dipertahankan dan sisa populasi diisi anak hasil seleksi, *crossover*, dan mutasi. Saat dijalankan dari menu main.py, jumlah populasi dan banyak iterasi dimasukkan oleh pengguna, sedangkan parameter lainnya memakai nilai bawaan pada definisi fungsi.
Output	*Dictionary* berisi *state* awal dan akhir, riwayat *fitness* maksimum dan rata-rata populasi, jumlah populasi, banyak iterasi, durasi, dan status *fallback*.

Source Code	`def genetic_algorithm(`
	`  problem,`
	`  rng,`
	`  population_size=50,`
	`  iterations=100,`
	`  p=0.3,`
	`  crossover_rate=0.9,`
	`  mutation_rate=0.3,`
	`  elite=1,`
	`):`
	`  start_time = time.perf_counter()`

```
    population, used_fallback =
make_population(problem, population_size, rng,
p)
    scores = [objective(s) for s in population]

    initial_state =
population[best_index(scores)]
    history = [max(scores)]
    history_avg = [sum(scores) /
population_size]

    for _ in range(iterations):
        ranked = sorted(range(population_size),
key=scores.__getitem__, reverse=True)
        next_population = [population[k] for k
in ranked[:elite]]
        next_scores = [scores[k] for k in
ranked[:elite]]
        while len(next_population) <
population_size:
            a = roulette(population, scores,
rng)
            b = roulette(population, scores,
rng)
            if rng.random() < crossover_rate:
                child = crossover(a, b, rng)
            else:
                child = a.copy()
            child = mutate(child,
mutation_rate, rng)
            next_population.append(child)

next_scores.append(objective(child))
        population, scores = next_population,
next_scores
        history.append(max(scores))
        history_avg.append(sum(scores) /
population_size)

    return {
        "algorithm": "ga",
        "initial_state": initial_state,
```

	`"final_state": population[best_index(scores)], "history": history, "history_avg": history_avg, "population_size": population_size, "iterations": iterations, "duration_s": time.perf_counter() - start_time, "used_fallback": used_fallback, }`

Nama Fungsi	`make_population(problem, size, rng, p)`
Deskripsi	Membangkitkan populasi awal berisi sejumlah size individu valid secara acak.
Output	Pasangan (daftar individu, status *fallback*).
Source Code	`def make_population(problem, size, rng, p): population = [] used_fallback = False for _ in range(size): state, fallback = random_initial_state(problem, rng, p) population.append(state) used_fallback = used_fallback or fallback return population, used_fallback`

Nama Fungsi	`roulette(population, scores, rng)`
Deskripsi	Memilih satu induk dengan seleksi *roulette wheel*, yaitu peluang terpilih sebanding dengan nilai *fitness*. Jika total *fitness* tidak positif, induk dipilih secara acak.
Output	Satu individu yang terpilih sebagai induk.
Source Code	`def roulette(population, scores, rng): total = sum(scores)`

	`if total <= 0:`
	`  return rng.choice(population)`
	`target = rng.uniform(0, total)`
	`acc = 0.0`
	`for individual, score in zip(population, scores):`
	`  acc += score`
	`  if acc >= target:`
	`  return individual`
	`return population[-1]`

Nama Fungsi	`crossover(a, b, rng)`
Deskripsi	Membentuk satu anak dengan mengambil posisi setiap kendaraan dari salah satu induk secara acak. Jika posisi tersebut membuat anak tidak valid, posisi dari induk lainnya dicoba, dan jika tetap gagal kendaraan ditempatkan di luar kapal.
Output	*State* anak yang valid.

Source Code	`def crossover(a, b, rng):`
	`  child = State.empty(a.problem)`
	`  order = list(range(len(a.problem.vehicles)))`
	`  rng.shuffle(order)`
	`  for i in order:`
	`  parents = [a, b] if rng.random() < 0.5 else [b, a]`
	`  for parent in parents:`
	`  if not parent.is_inside(i):`
	`  continue`
	`  child.positions[i] = parent.positions[i]`
	`  child.orientations[i] = parent.orientations[i]`
	`  if is_valid(child):`
	`  break`
	`  child.positions[i] = None`
	`  child.orientations[i] = None`
	`  return child`

Nama Fungsi	`mutate(state, rate, rng)`
Deskripsi	Dengan peluang rate, mengganti *state* dengan satu *neighbor* acak. Jika *neighbor* tidak valid, *state* tidak berubah.
Output	*State* hasil mutasi atau *state* semula.

Source Code	`def mutate(state, rate, rng):`
	`  if rng.random() >= rate:`
	`  return state`
	`  _, neighbor = get_random_neighbor(state, rng)`
	`  return neighbor if neighbor is not None else state`

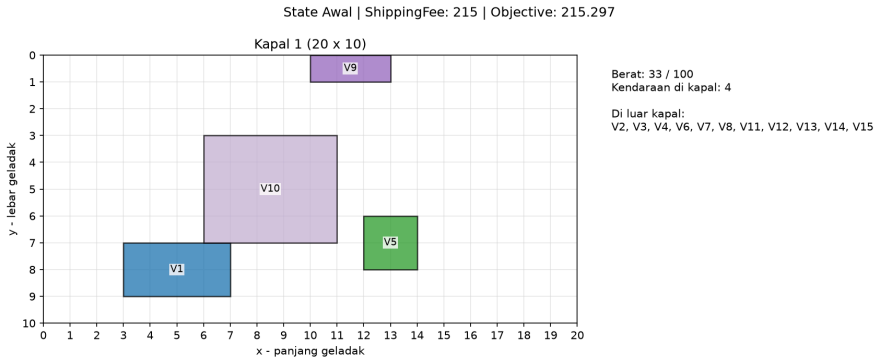
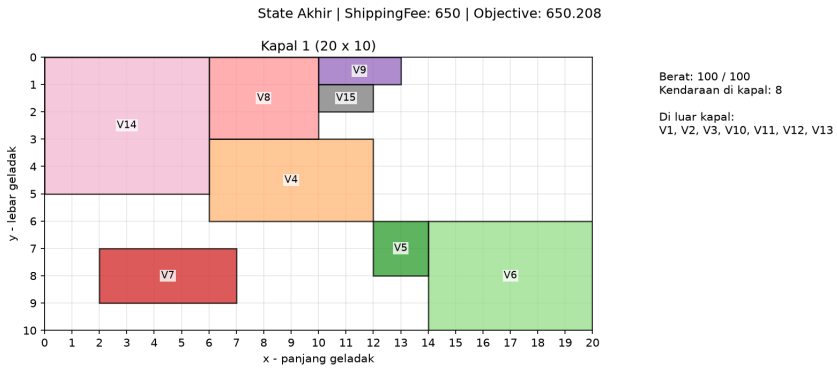
Nama Fungsi	`best_index(scores)`
Deskripsi	Mencari indeks individu dengan nilai *fitness* tertinggi.
Output	Indeks individu terbaik.
Source Code	`def best_index(scores):` `  return max(range(len(scores)), key=scores.__getitem__)`

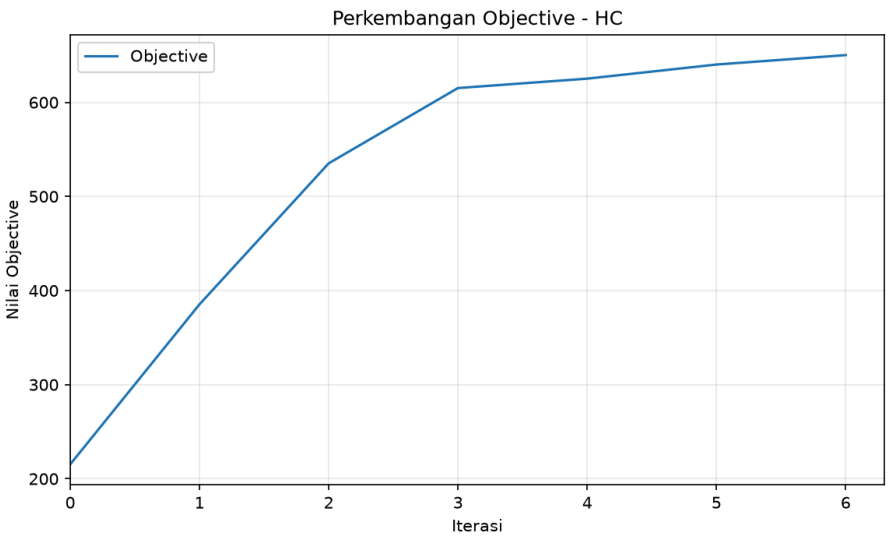
2.3. Hasil Eksperimen dan Analisis

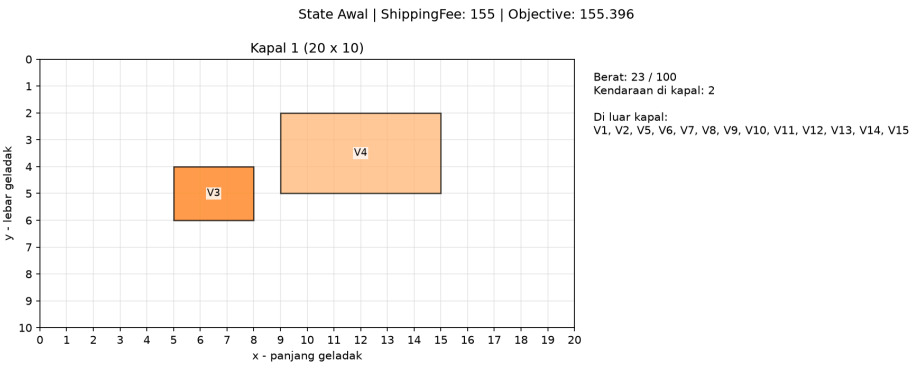
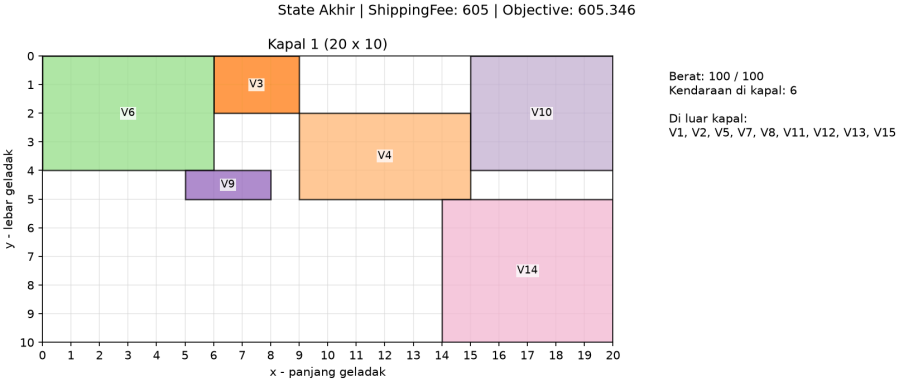
2.3.1. Algoritma *Steepest Ascent Hill-Climbing*

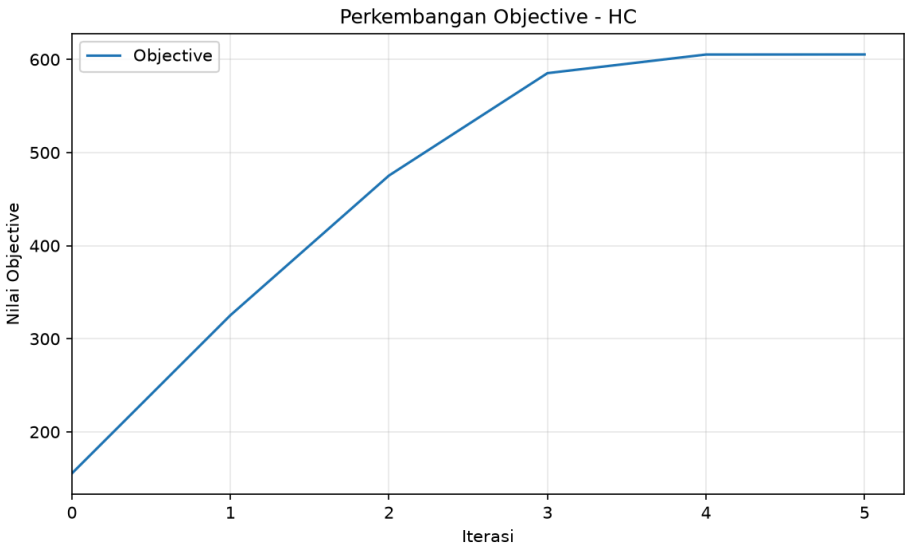
Berikut merupakan hasil tiga kali eksperimen menggunakan algoritma Steepest Ascent Hill-Climbing pada data sedang.json.

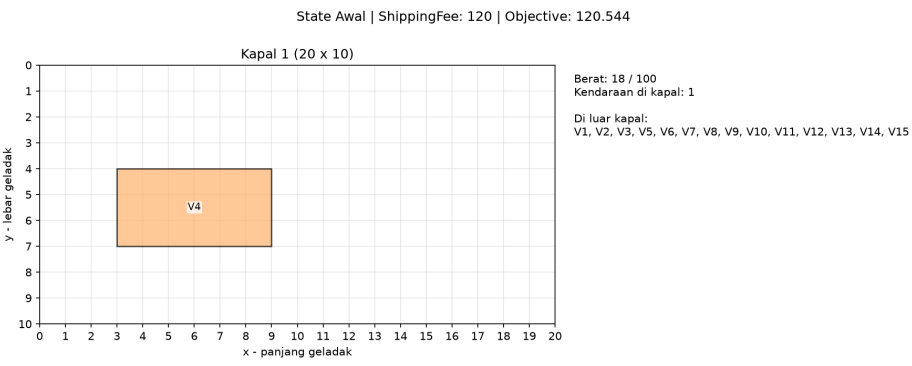
Eksperimen 1

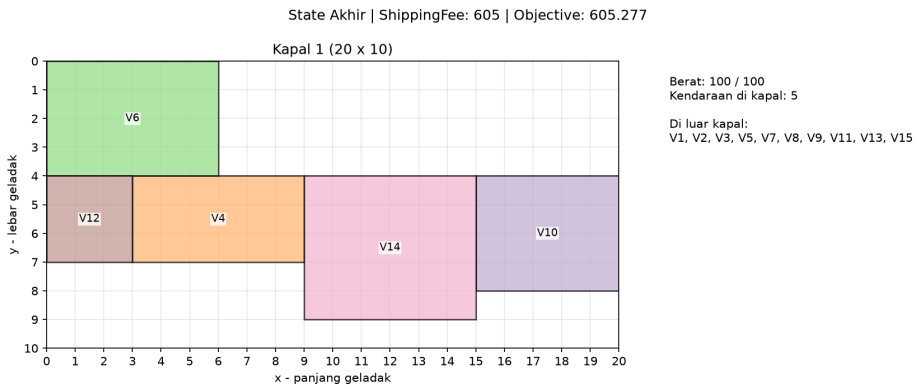
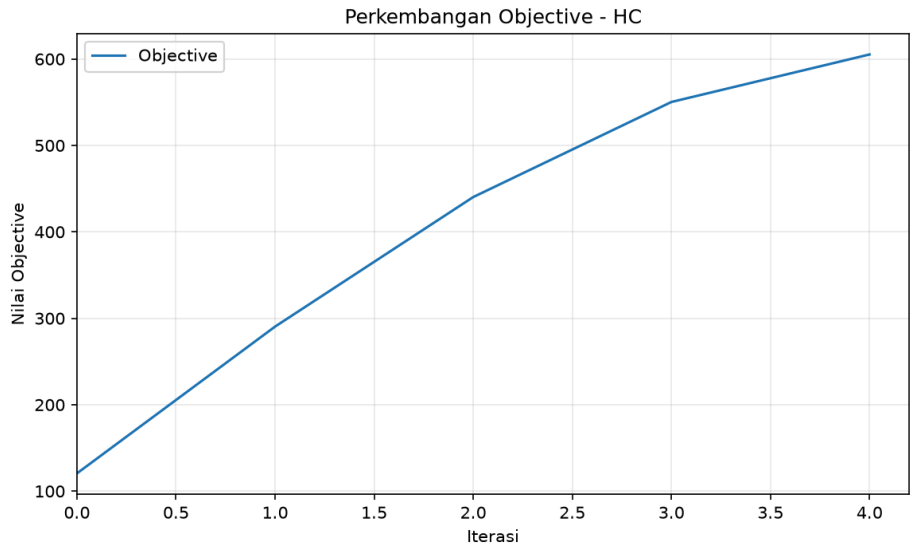
State awal	
State akhir	
Nilai objective awal	215.2970
Nilai objective akhir	650.2079
Total ShippingFee akhir	650
Kendaraan dimuat (awal → akhir)	4 → 8
Durasi	0.2569 s
Banyak iterasi	6

Plot objective	

Eksperimen 2	
State awal	
State akhir	
Nilai objective awal	155.3960
Nilai objective	605.3465

akhir	
Total ShippingFee akhir	605
Kendaraan dimuat (awal → akhir)	2 → 6
Durasi	0.3115 s
Banyak iterasi	5
Plot objective	

Eksperimen 3	
State awal	

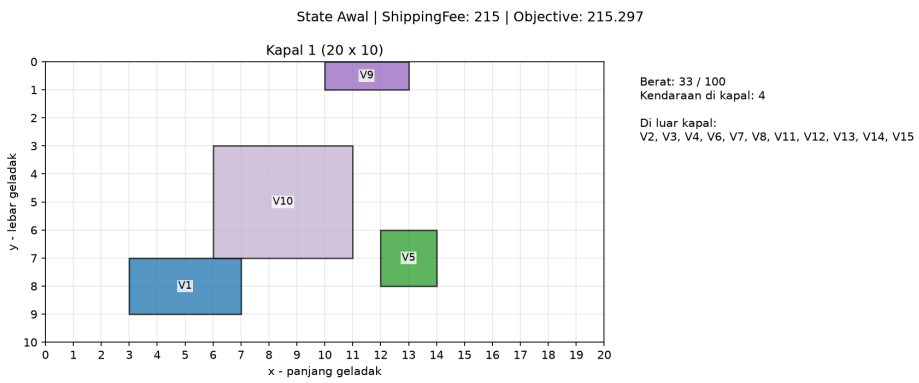
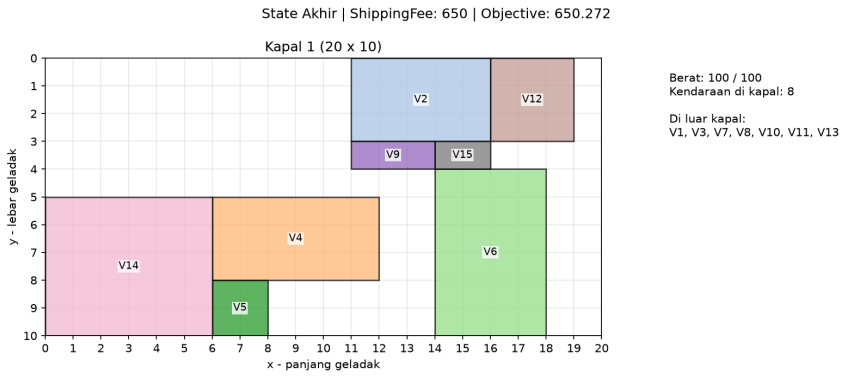
State akhir	
Nilai objective awal	120.5445
Nilai objective akhir	605.2772
Total ShippingFee akhir	605
Kendaraan dimuat (awal → akhir)	1 → 5
Durasi	0.3766 s
Banyak iterasi	4
Plot objective	

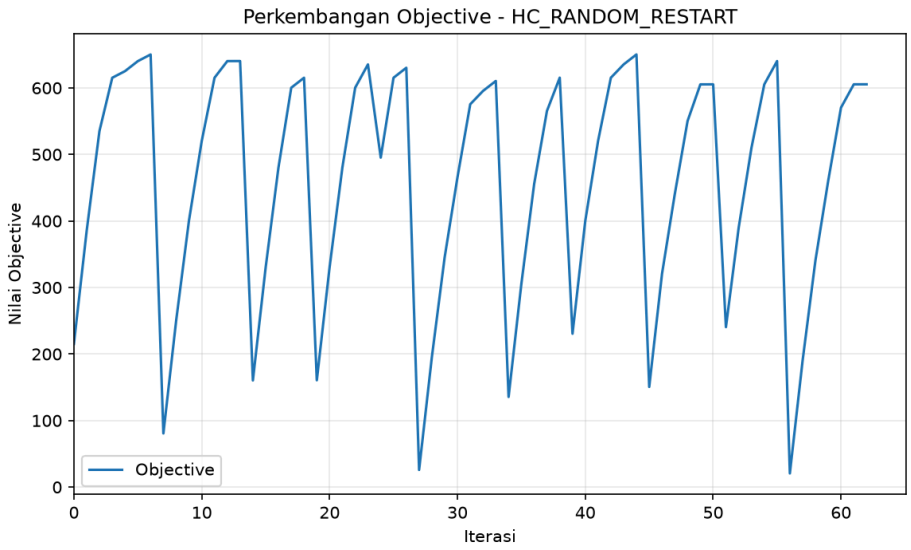
Berdasarkan ketiga eksperimen, algoritma Steepest Ascent Hill-Climbing berhenti dengan sangat cepat, yaitu hanya dalam 4 hingga 6 iterasi. Hal ini terjadi karena pada setiap iterasi algoritma langsung memilih *neighbor* dengan nilai *objective* terbaik, sehingga kenaikan nilai

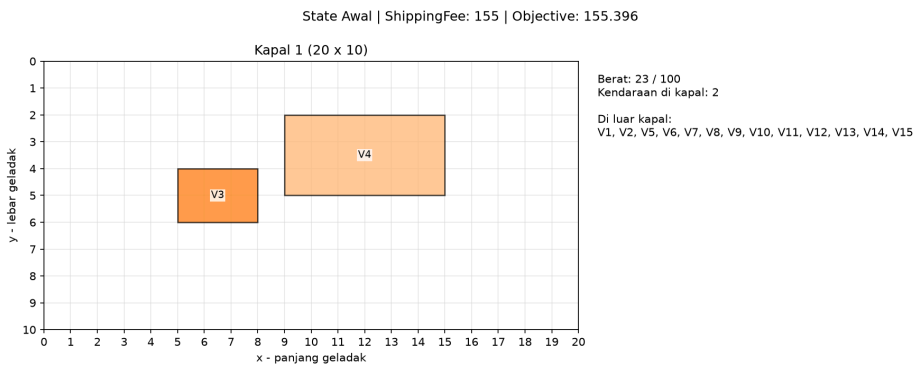
terjadi dengan cepat hingga tidak ada lagi *neighbor* yang lebih baik dari *state* saat ini, yaitu ketika algoritma mencapai *local maximum*. Hasil akhir yang diperoleh juga bergantung pada *state* awal. Eksperimen 1 berakhir dengan total *ShippingFee* 650, sedangkan eksperimen 2 dan 3 sama-sama berhenti di 605, yang menunjukkan bahwa algoritma terjebak pada puncak yang berbeda sesuai titik awalnya. Meskipun jumlah iterasinya sedikit, setiap iterasi tetap membutuhkan waktu karena algoritma harus membangkitkan dan mengevaluasi seluruh *neighbor* yang valid dari hasil *swap*, *shift*, *rotate*, *remove*, dan *insert*, sehingga durasi pencarian berada pada kisaran 0,26 hingga 0,38 detik.

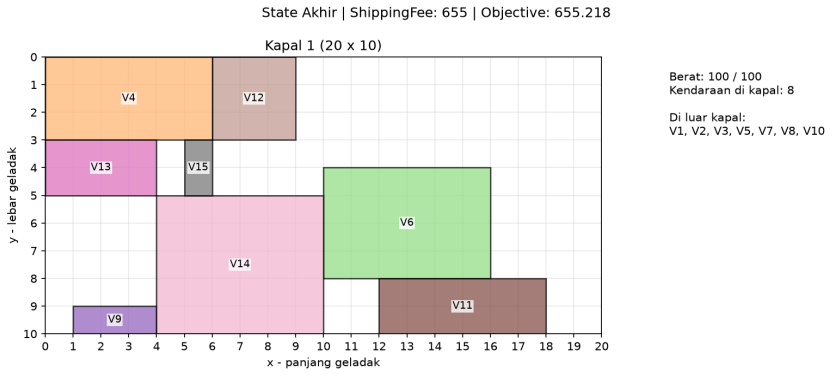
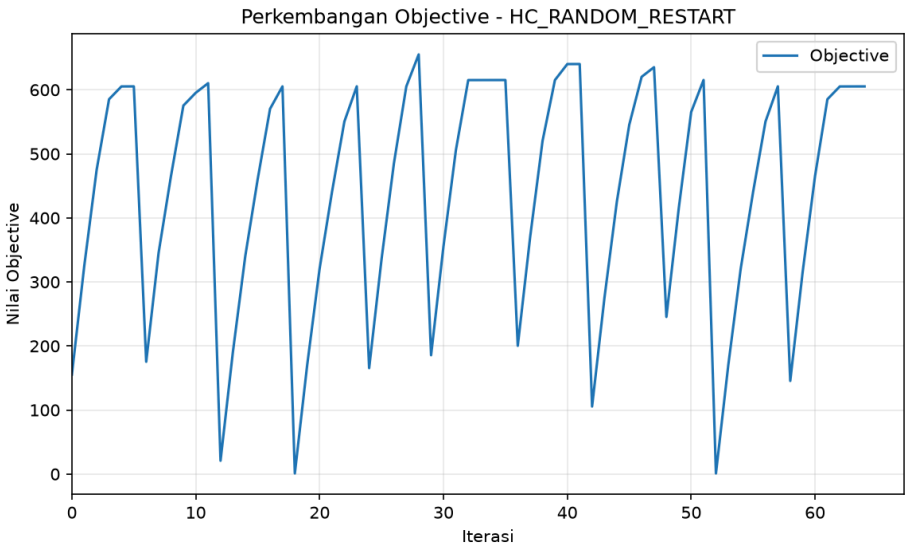
2.3.2. Algoritma *Random Restart Hill-Climbing*

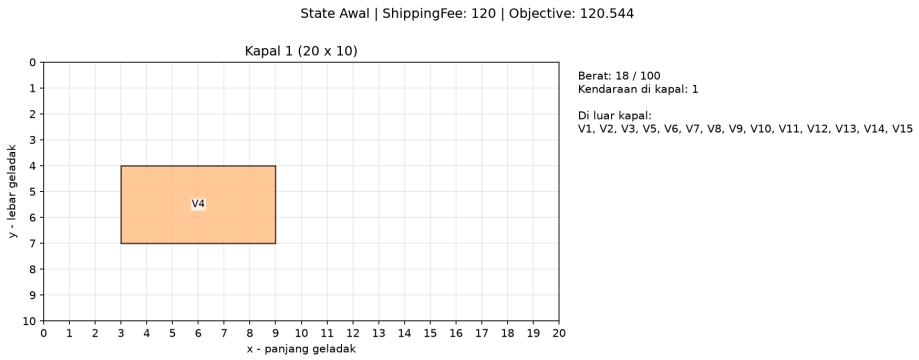
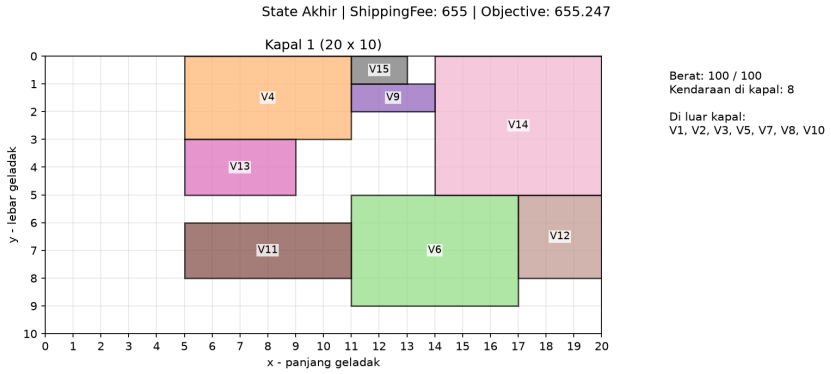
Berikut merupakan hasil tiga kali eksperimen menggunakan algoritma Random Restart Hill-Climbing dengan maksimum restart sebanyak 10 pada data sedang.json.

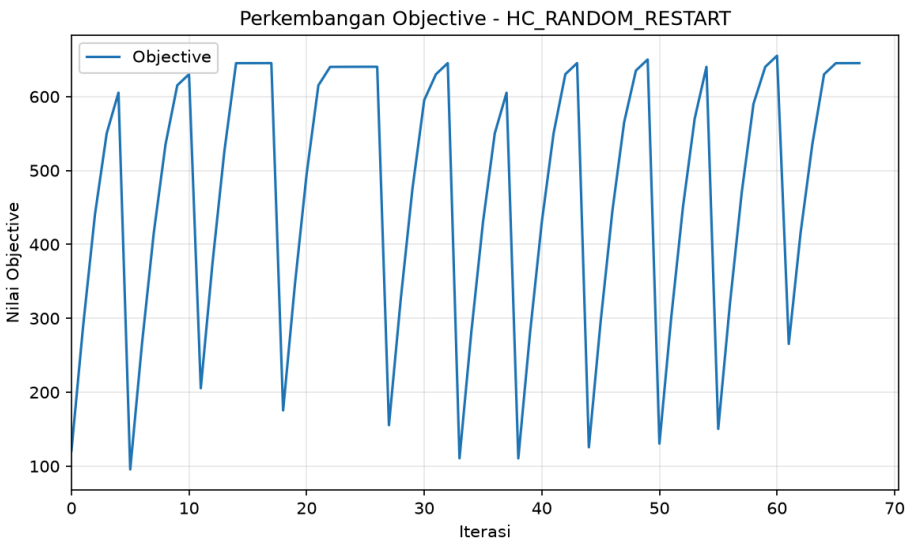
<table>
<tr><th colspan="2">Eksperimen 1</th></tr>
<tr><td>State awal</td><td> 
</td></tr>
<tr><td>State akhir</td><td> 
</td></tr>
<tr><td>Nilai objective awal</td><td>215.2970</td></tr>
<tr><td>Nilai objective akhir</td><td>650.2722</td></tr>
<tr><td>Total ShippingFee akhir</td><td>650</td></tr>
</table>

Kendaraan dimuat (awal → akhir)	4 → 8
Durasi	3.7730 s
Banyak restart	10
Banyak iterasi per restart	6, 6, 4, 4, 2, 6, 4, 5, 5, 4, 6
Total iterasi	52
Plot objective	

Eksperimen 2	
State awal	

State akhir	
Nilai objective awal	155.3960
Nilai objective akhir	655.2178
Total ShippingFee akhir	655
Kendaraan dimuat (awal → akhir)	2 → 8
Durasi	4.2029 s
Banyak restart	10
Banyak iterasi per restart	5, 5, 5, 5, 4, 6, 5, 5, 3, 5, 6
Total iterasi	54
Plot objective	

Eksperimen 3	
State awal	
State akhir	
Nilai objective awal	120.5445
Nilai objective akhir	655.2475
Total ShippingFee akhir	655
Kendaraan dimuat (awal → akhir)	1 → 8
Durasi	3.4603 s
Banyak restart	10
Banyak iterasi per restart	4, 5, 6, 8, 5, 4, 5, 5, 4, 5, 6
Total iterasi	57

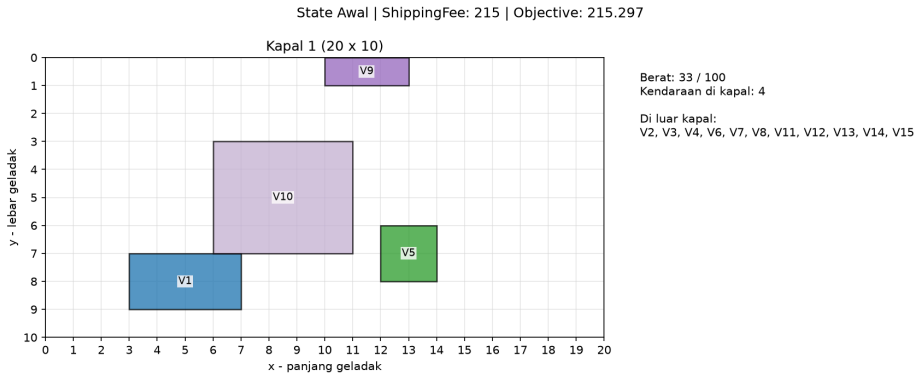
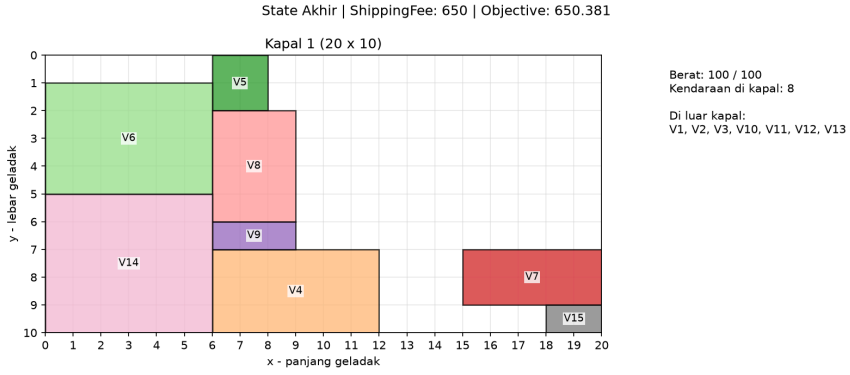
Plot objective	

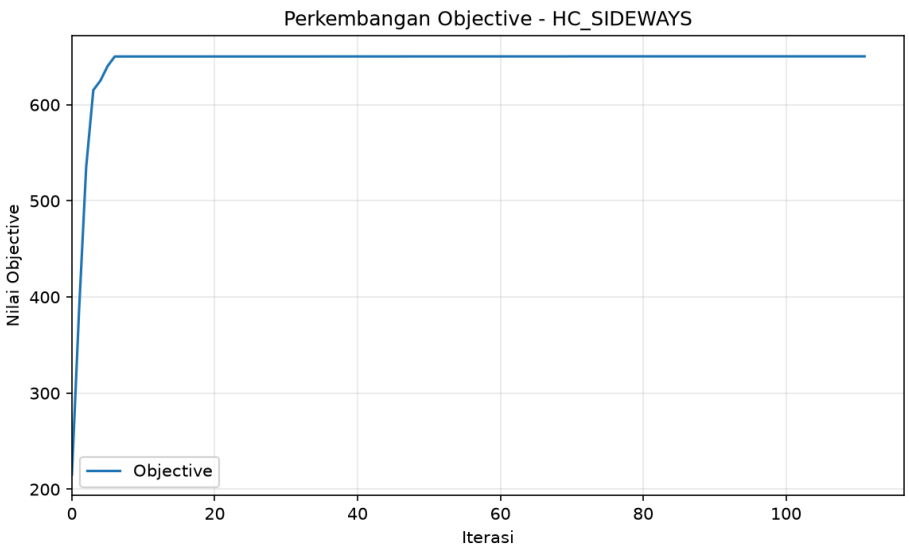
Algoritma Random Restart Hill-Climbing menjalankan Steepest Ascent sebanyak sebelas kali (satu pencarian awal dan sepuluh *restart*) dari *state* awal acak yang berbeda, lalu mengambil hasil terbaik. Setiap pencarian hanya membutuhkan 2 hingga 8 iterasi, yang menunjukkan bahwa setiap pencarian tetap cepat terjebak pada *local maximum*. Namun, dengan mencoba banyak titik awal, peluang menemukan puncak yang lebih tinggi menjadi lebih besar. Hal ini terlihat pada eksperimen 2 dan 3, yang total *ShippingFee*-nya naik dari 605 pada Steepest Ascent menjadi 655, sedangkan eksperimen 1 tetap di 650. Hasil ketiga eksperimen juga lebih konsisten (650–655), sehingga algoritma ini tidak terlalu bergantung pada satu *state* awal. Kelemahannya terletak pada durasi, yaitu 3,46 hingga 4,20 detik, atau sekitar sepuluh kali lebih lama dibandingkan Steepest Ascent, karena proses pencarian diulang berkali-kali.

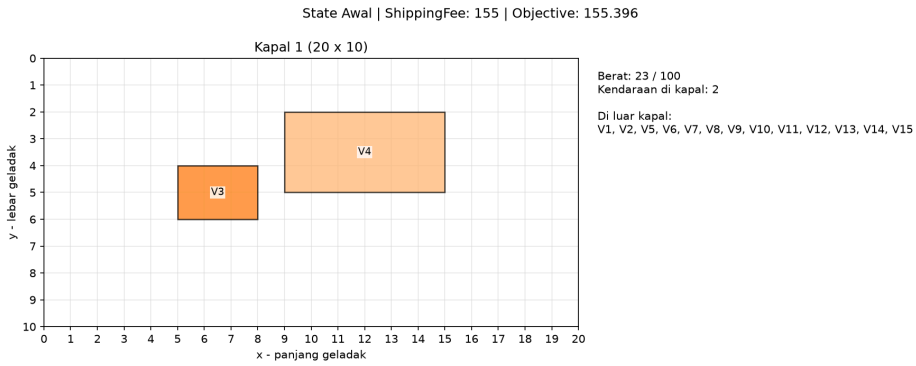
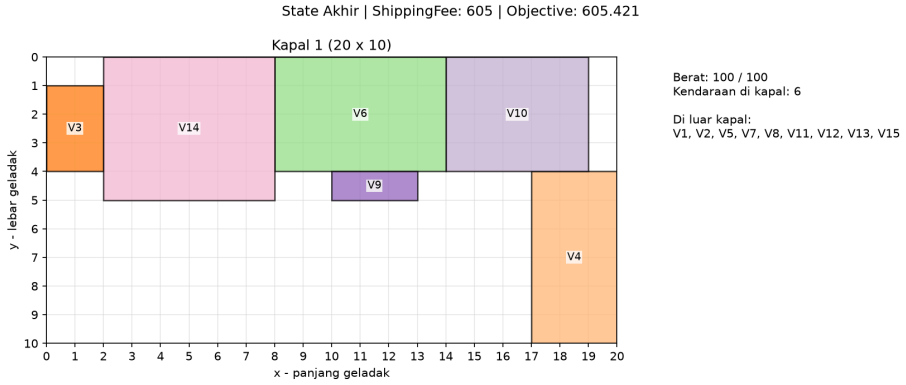
2.3.3. Algoritma *Sideways Hill-Climbing*

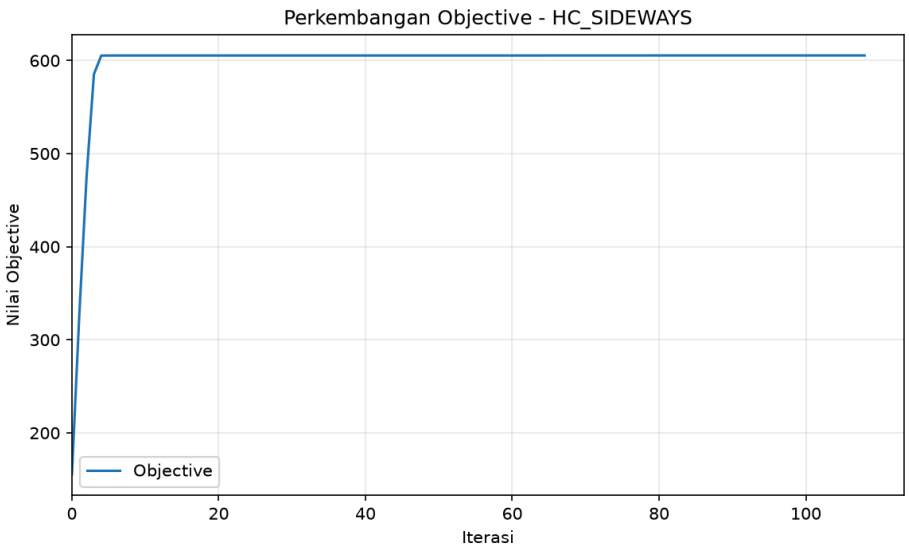
Berikut merupakan hasil tiga kali eksperimen menggunakan algoritma Hill-Climbing with Sideways Move dengan maksimum sideways move sebanyak 100 pada data sedang.json.

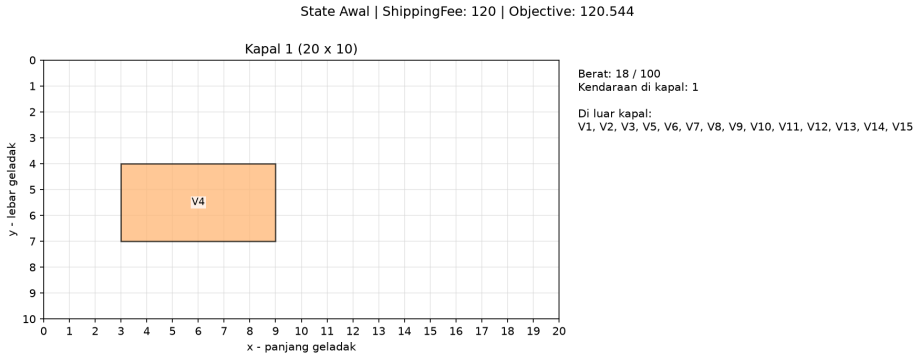
Eksperimen 1

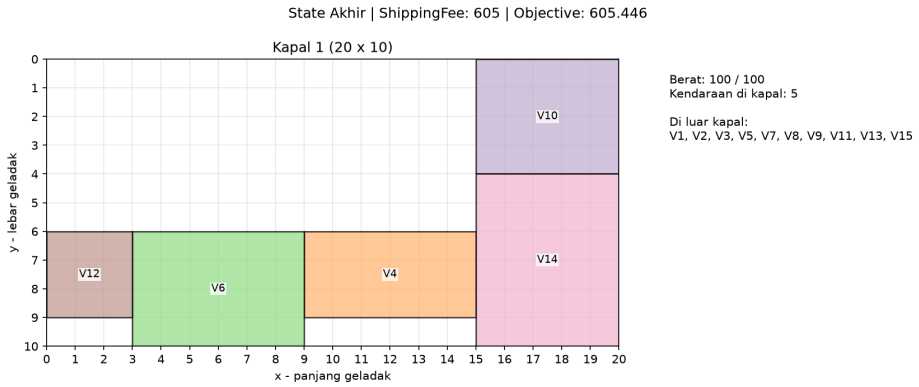
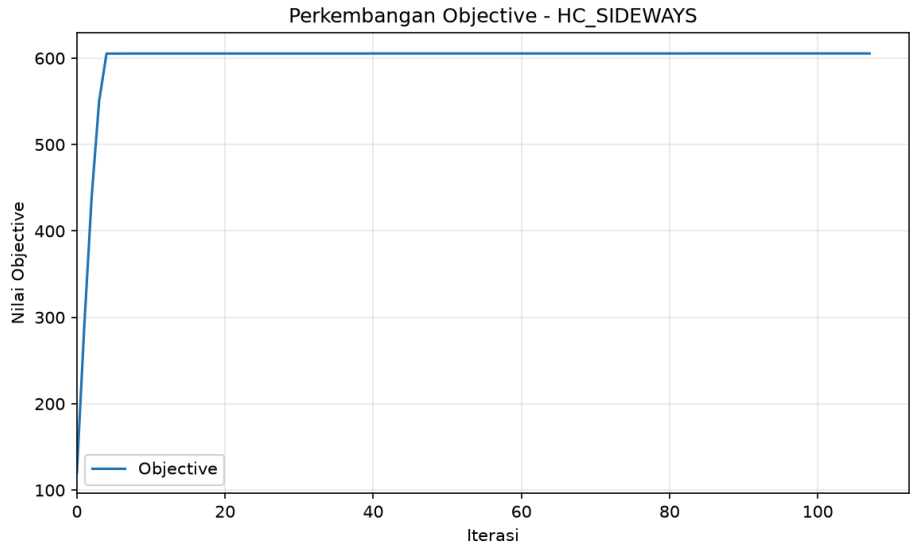
State awal	
State akhir	
Nilai objective awal	215.2970
Nilai objective akhir	650.3812
Total ShippingFee akhir	650
Kendaraan dimuat (awal → akhir)	4 → 8
Durasi	1.9120 s
Banyak iterasi	111
Banyak sideways move	100

Plot objective	

Eksperimen 2	
State awal	
State akhir	
Nilai objective awal	155.3960
Nilai objective	605.4207

akhir	
Total ShippingFee akhir	605
Kendaraan dimuat (awal → akhir)	2 → 6
Durasi	2.4869 s
Banyak iterasi	108
Banyak sideways move	100
Plot objective	

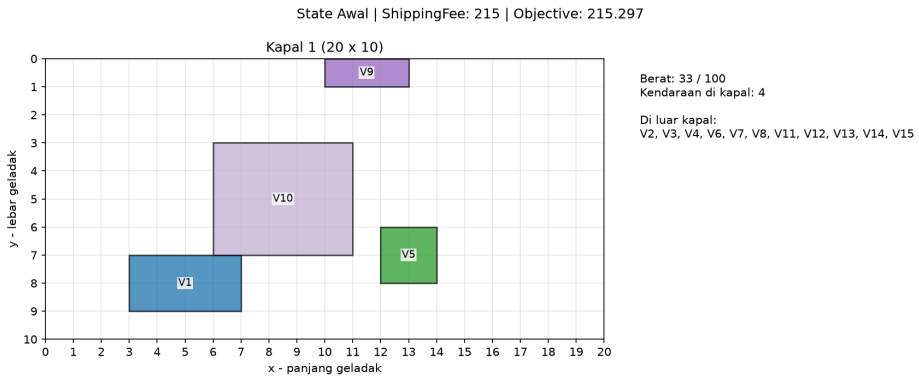
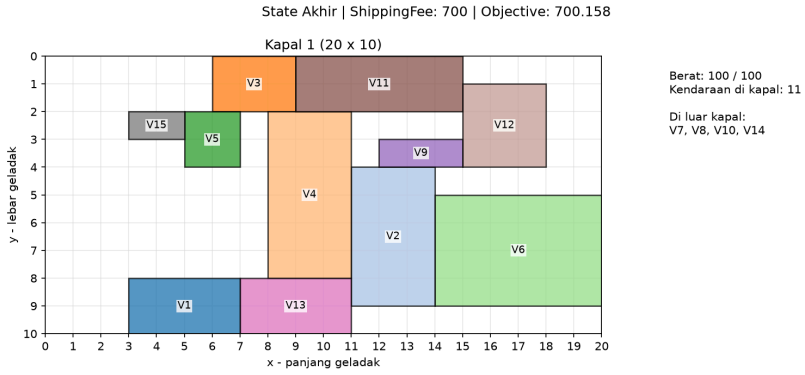
Eksperimen 3	
State awal	

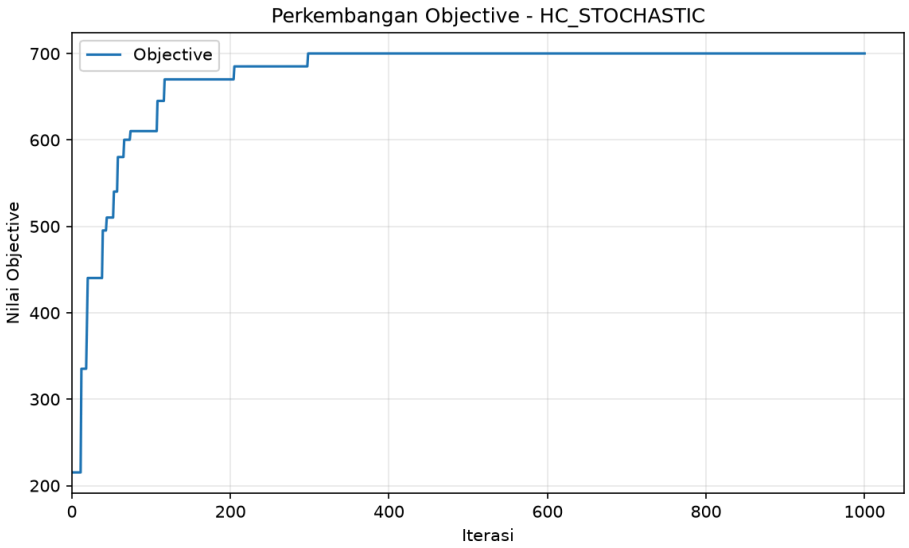
State akhir	
Nilai objective awal	120.5445
Nilai objective akhir	605.4455
Total ShippingFee akhir	605
Kendaraan dimuat (awal → akhir)	1 → 5
Durasi	2.6000 s
Banyak iterasi	107
Banyak sideways move	100
Plot objective	

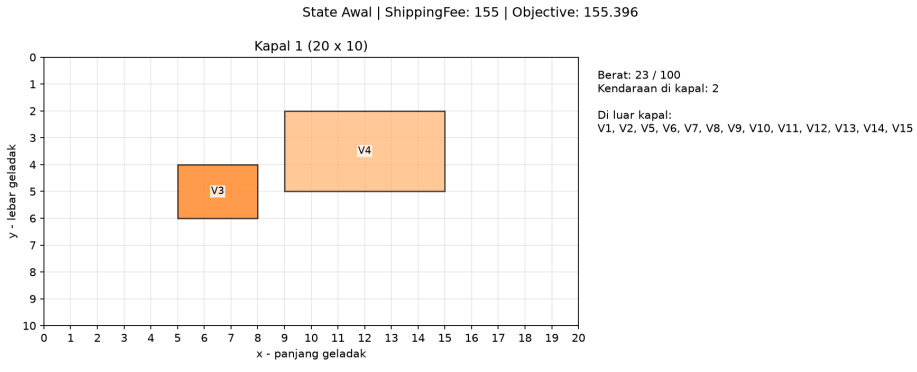
Pada ketiga eksperimen, Hill-Climbing with Sideways Move menghasilkan total *ShippingFee* yang sama persis dengan Steepest Ascent, yaitu 650, 605, dan 605. Perbedaannya hanya terdapat pada bagian desimal nilai *objective*, yang menandakan bahwa susunan akhir memiliki ruang kosong yang lebih lega. Seluruh eksperimen menghabiskan batas 100 *sideways move*, sehingga jumlah iterasinya naik menjadi 107 hingga 111. Hal ini menunjukkan bahwa algoritma berpindah-pindah di antara *state* dengan nilai yang sama (*plateau*), tetapi tidak menemukan jalan menuju *state* dengan *ShippingFee* yang lebih tinggi sebelum batas *sideways move* habis. Akibatnya, tambahan waktu yang cukup besar, yaitu 1,91 hingga 2,60 detik, tidak sebanding dengan peningkatan kualitas solusi yang didapat.

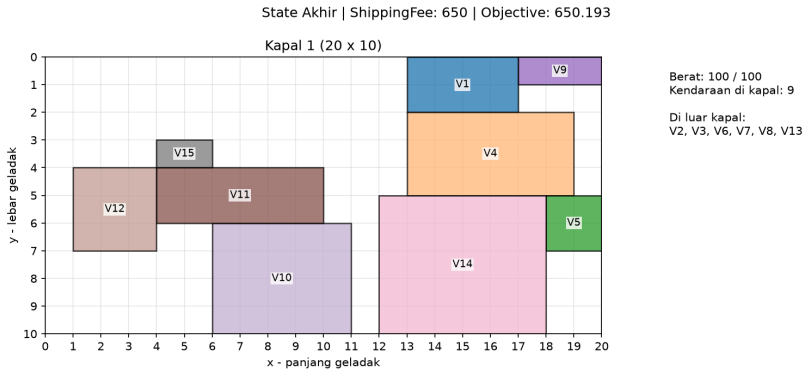
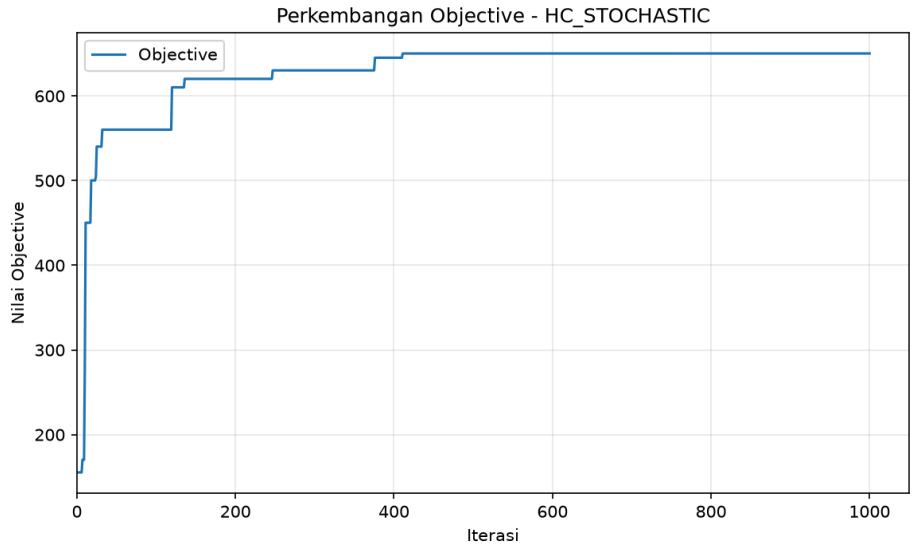
2.3.4. Algoritma *Stochastic Hill-Climbing*

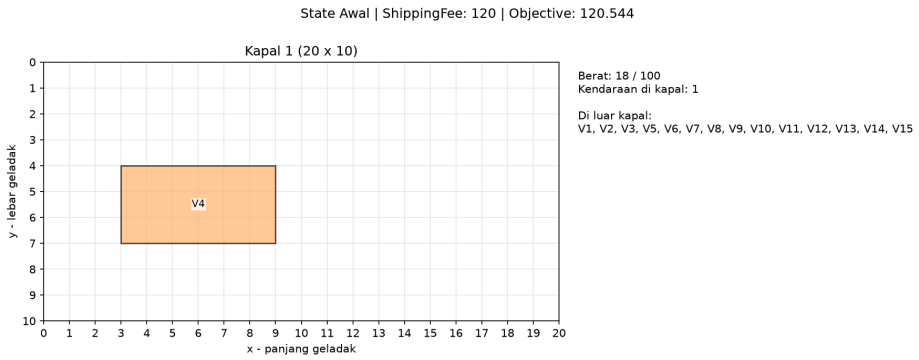
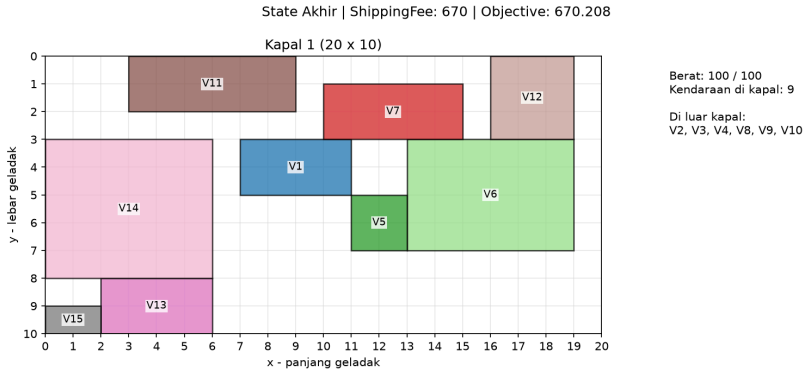
Berikut merupakan hasil tiga kali eksperimen menggunakan algoritma Stochastic Hill-Climbing dengan maksimum iterasi sebanyak 1000 pada data sedang.json.

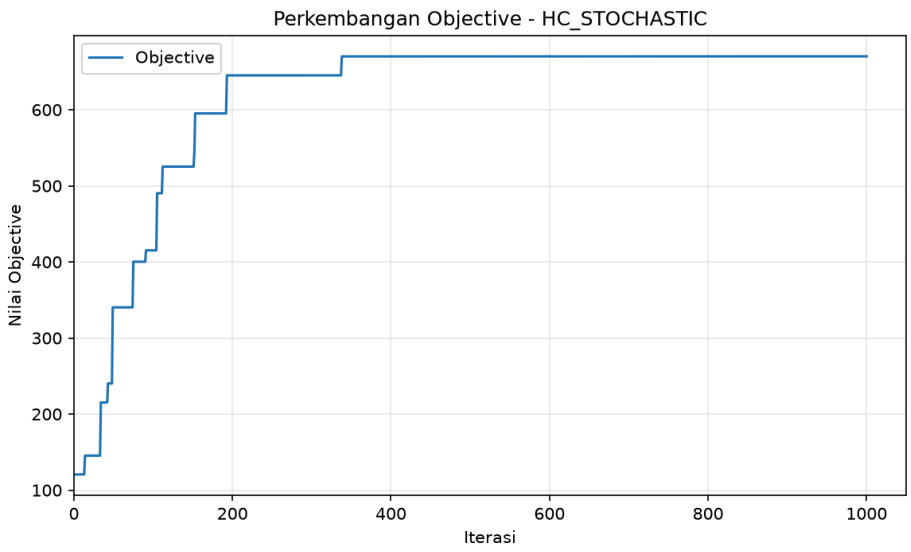
<table>
<tr><th colspan="2">Eksperimen 1</th></tr>
<tr><td>State awal</td><td> 
</td></tr>
<tr><td>State akhir</td><td> 
</td></tr>
<tr><td>Nilai objective awal</td><td>215.2970</td></tr>
<tr><td>Nilai objective akhir</td><td>700.1584</td></tr>
</table>

Total ShippingFee akhir	700
Kendaraan dimuat (awal → akhir)	4 → 11
Durasi	0.0352 s
Banyak iterasi	1000
Neighbor tidak valid	609
Plot objective	

Eksperimen 2	
State awal	

State akhir	
Nilai objective awal	155.3960
Nilai objective akhir	650.1930
Total ShippingFee akhir	650
Kendaraan dimuat (awal → akhir)	2 → 9
Durasi	0.0338 s
Banyak iterasi	1000
Neighbor tidak valid	599
Plot objective	

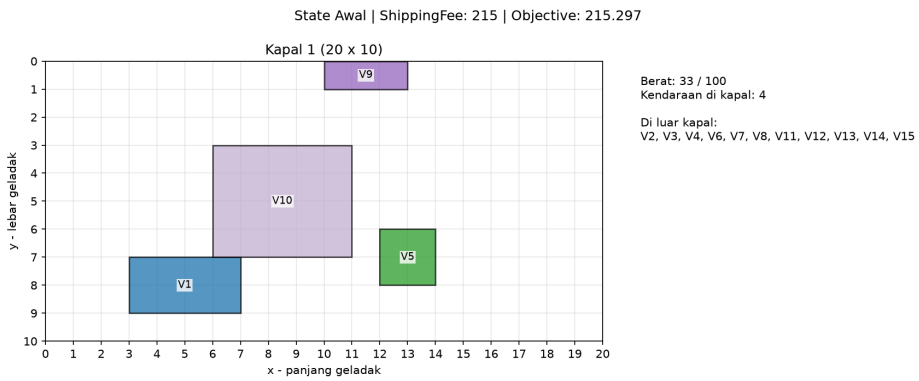
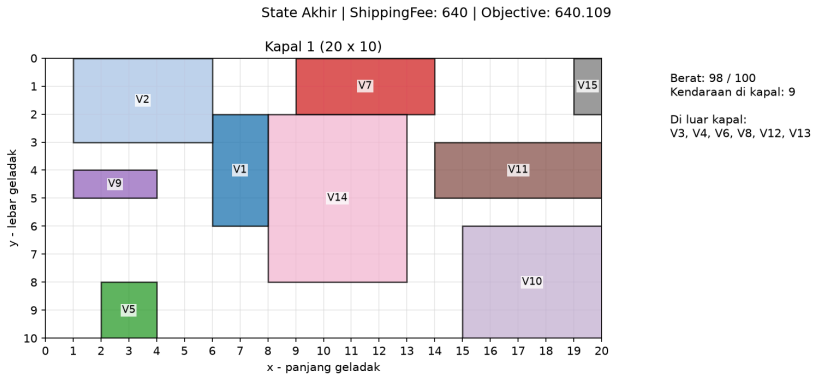
Eksperimen 3	
State awal	
State akhir	
Nilai objective awal	120.5445
Nilai objective akhir	670.2079
Total ShippingFee akhir	670
Kendaraan dimuat (awal → akhir)	1 → 9
Durasi	0.0403 s
Banyak iterasi	1000
Neighbor tidak valid	528

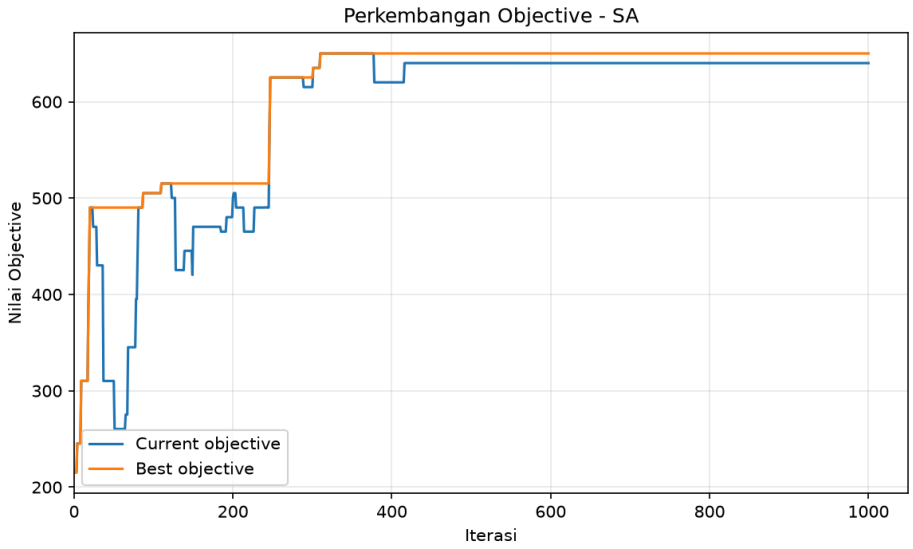
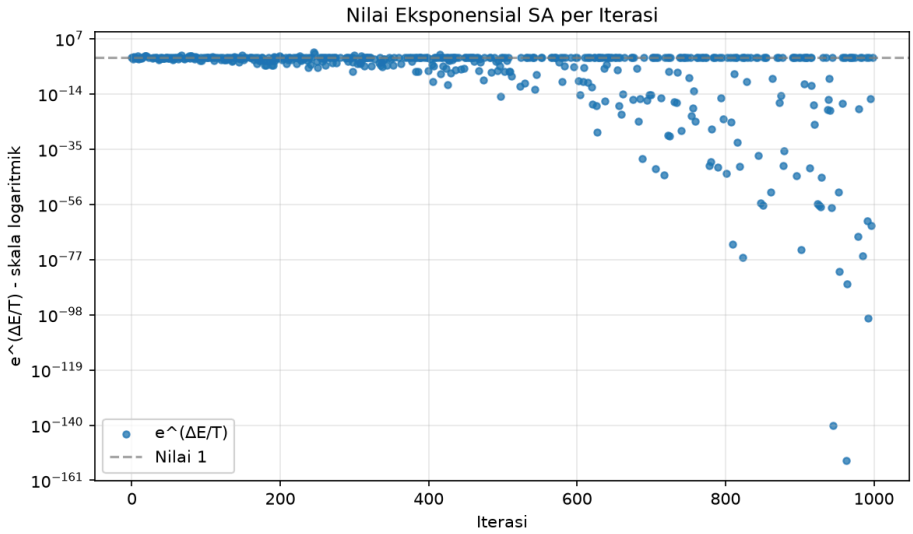
Plot objective	

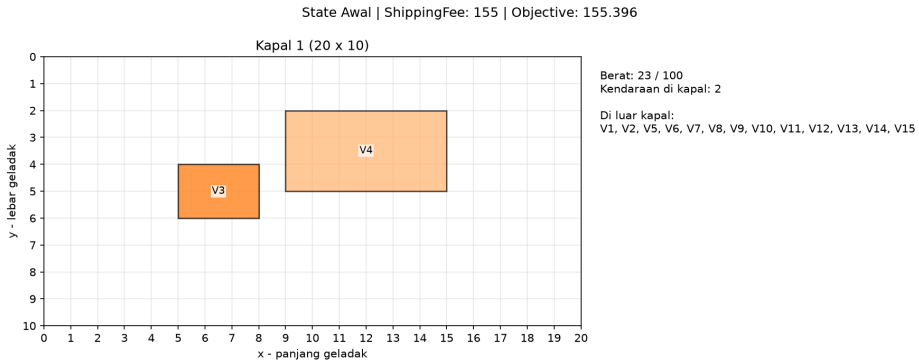
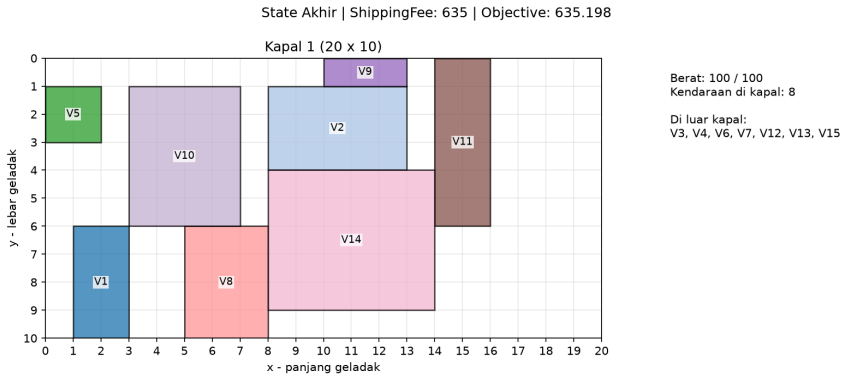
Stochastic Hill-Climbing memberikan hasil terbaik di antara seluruh varian Hill-Climbing pada ketiga eksperimen, dengan total *ShippingFee* 700, 650, dan 670, sekaligus menjadi yang tercepat dengan durasi hanya sekitar 0,034 hingga 0,040 detik. Kecepatan ini diperoleh karena setiap iterasi hanya membangkitkan satu *neighbor* acak, bukan seluruh *neighbor* seperti Steepest Ascent. Dari 1000 iterasi, sebanyak 528 hingga 609 *neighbor* tidak valid dan langsung dilewati. Pada semua eksperimen, baik Steepest Ascent maupun Stochastic sama-sama berhenti ketika total berat kendaraan telah mencapai batas *MaxCapacity* (100), sehingga batasan yang paling menentukan adalah berat, bukan luas geladak. Perbedaannya, Steepest Ascent selalu mengambil kenaikan *ShippingFee* terbesar sehingga cenderung cepat memasukkan kendaraan besar yang menghabiskan kapasitas berat, sedangkan Stochastic menerima kenaikan sekecil apa pun sehingga kapasitas yang sama terisi oleh lebih banyak kendaraan yang lebih kecil, yaitu 9 hingga 11 kendaraan dibandingkan 5 hingga 8 kendaraan pada Steepest Ascent, dengan total *ShippingFee* yang lebih tinggi.

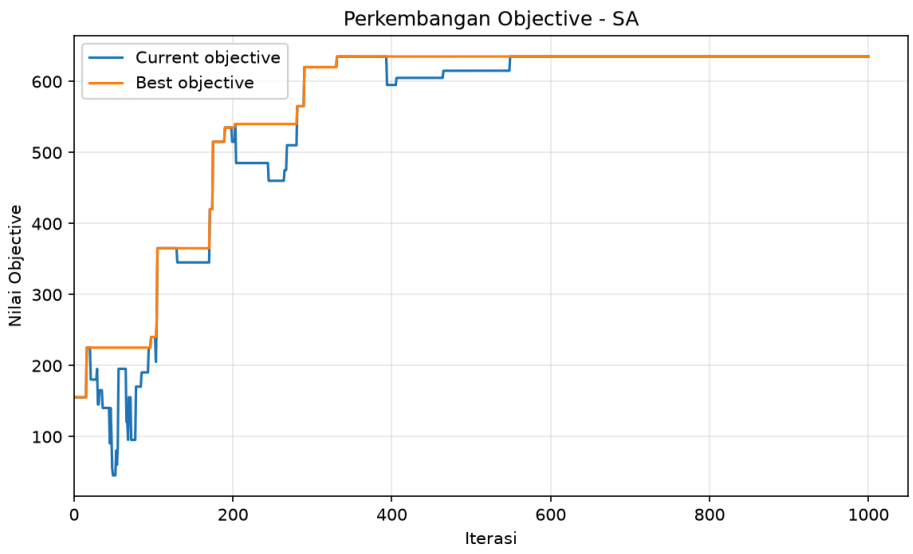
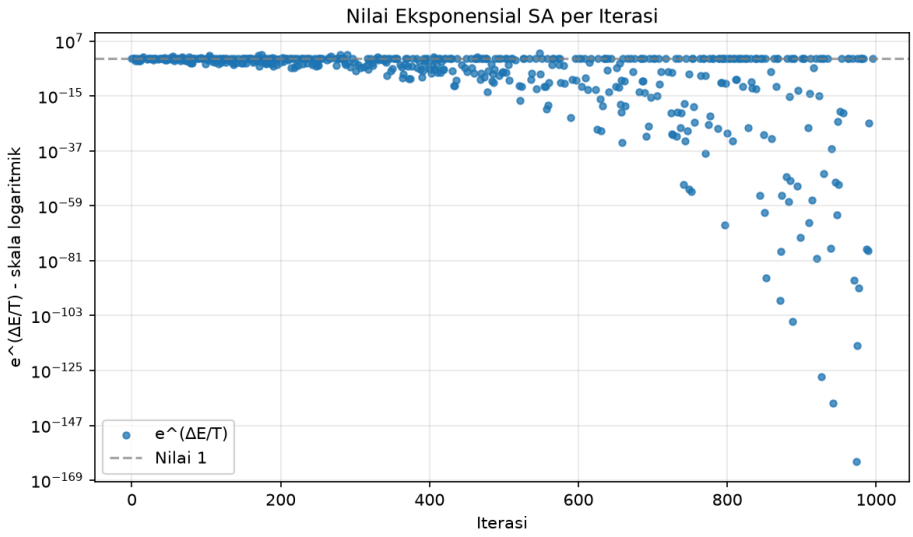
2.3.5. Algoritma *Simulated Annealing*

Berikut merupakan hasil tiga kali eksperimen menggunakan algoritma Simulated Annealing pada dataset sedang.json, dengan seed 1, 2, dan 3. Seluruh eksperimen menggunakan konfigurasi parameter yang sama, yaitu temperatur awal (*initial temperature*) sebesar 60, faktor pendinginan (*cooling rate*) sebesar 0,995, dan batas maksimum sebanyak 1.000 iterasi.

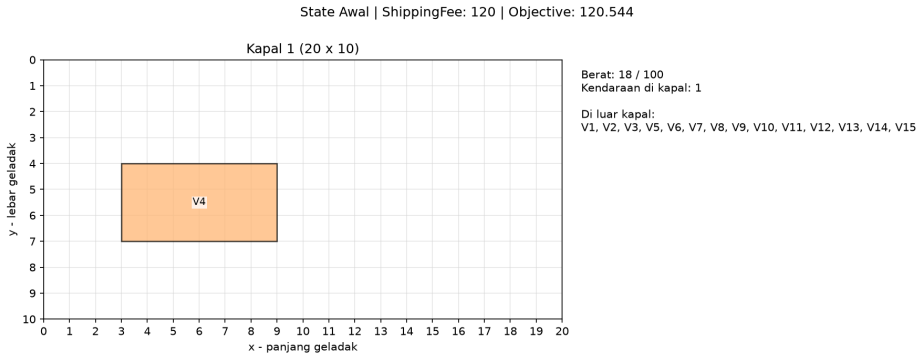
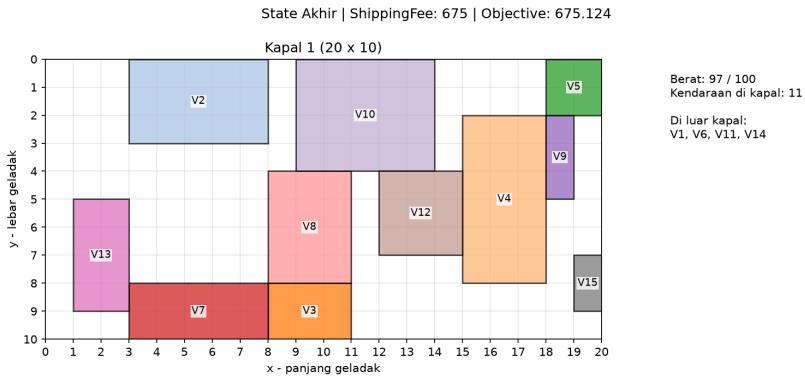
Eksperimen 1	
State awal	
State akhir	
Nilai objective awal	215.2970
Nilai objective akhir	640.1089
Total ShippingFee akhir	640
Kendaraan dimuat (awal → akhir)	4 → 9
Durasi	0.1066 s

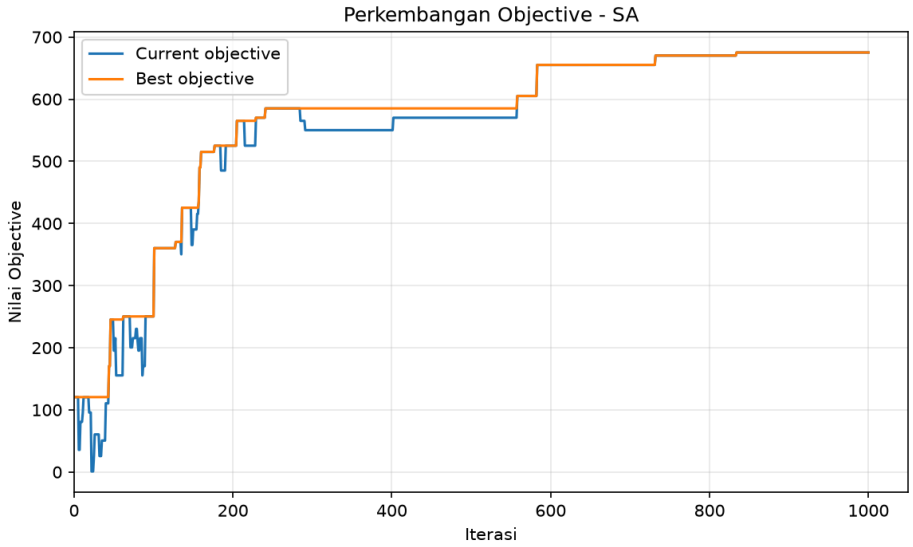
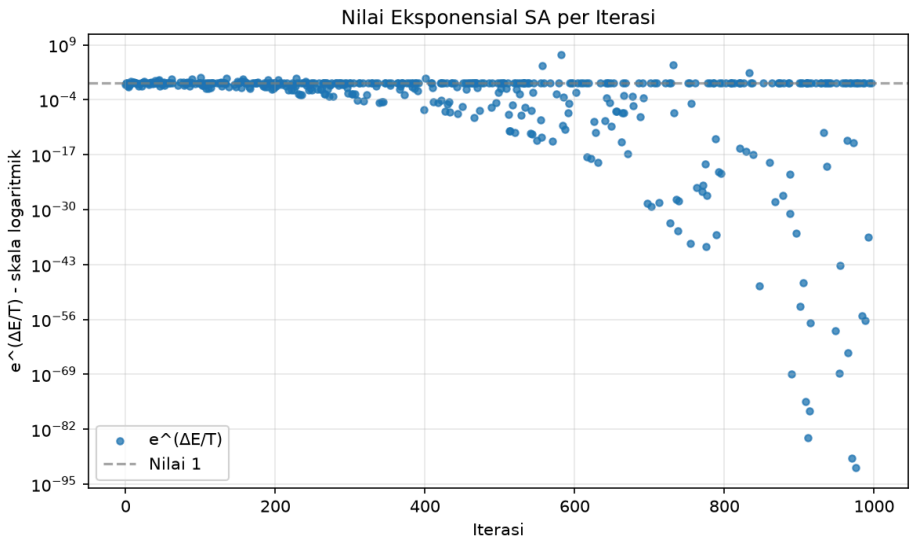
Banyak iterasi	1000
Neighbor tidak valid	481
Accepted worse moves	37
Stuck episodes	0
Best ShippingFee	650
Best objective	650.1337
Plot objective	
Plot eksponensial e^(ΔE/T)	

Eksperimen 2	
State awal	
State akhir	
Nilai objective awal	155.3960
Nilai objective akhir	635.1980
Total ShippingFee akhir	635
Kendaraan dimuat (awal → akhir)	2 → 8
Durasi	0.1288 s
Banyak iterasi	1000
Neighbor tidak valid	448
Accepted worse	48

moves	
Stuck episodes	7
Best ShippingFee	635
Best objective	635.2475
Plot objective	
Plot eksponensial e^(ΔE/T)	

Eksperimen 3

State awal	
State akhir	
Nilai objective awal	120.5445
Nilai objective akhir	675.1238
Total ShippingFee akhir	675
Kendaraan dimuat (awal → akhir)	1 → 11
Durasi	0.1086 s
Banyak iterasi	1000
Neighbor tidak valid	517
Accepted worse moves	39
Stuck episodes	0
Best ShippingFee	675

Best objective	675.1485
Plot objective	
Plot eksponensial e^(ΔE/T)	

Berdasarkan ketiga eksperimen, algoritma Simulated Annealing menghasilkan nilai ShippingFee terbaik sebesar 650, 635, dan 675, dengan rata-rata sebesar 653,33. Hasil tertinggi diperoleh pada eksperimen ketiga dengan seed 3, yaitu ShippingFee sebesar 675 dan nilai objective sebesar 675,1485. Perbedaan hasil antareksperimen menunjukkan pengaruh pembangkitan state awal dan pemilihan neighbor secara acak terhadap kualitas solusi yang ditemukan.

Pada eksperimen pertama, state akhir menghasilkan ShippingFee sebesar 640, sedangkan best state yang pernah ditemukan menghasilkan ShippingFee sebesar 650. Perbedaan tersebut menunjukkan bahwa mekanisme penerimaan perpindahan menuju neighbor yang lebih buruk memungkinkan algoritma mengakhiri pencarian pada solusi yang nilainya lebih rendah daripada solusi terbaik yang pernah ditemukan. Oleh karena itu, penyimpanan best state diperlukan untuk mempertahankan solusi terbaik selama proses pencarian.

Selama 1.000 iterasi, ketiga eksperimen menghasilkan masing-masing 481, 448, dan 517 neighbor tidak valid. Secara keseluruhan, sebanyak 48,2% kandidat neighbor tidak memenuhi constraint sehingga langsung ditolak. Selain itu, algoritma menerima perpindahan menuju neighbor yang lebih buruk sebanyak 37, 48, dan 39 kali. Hasil ini menunjukkan bahwa mekanisme penerimaan probabilistik digunakan selama pencarian dan memungkinkan eksplorasi solusi yang tidak selalu meningkatkan nilai objective.

Berdasarkan mekanisme diagnosis local maximum, eksperimen pertama dan ketiga tidak menghasilkan episode yang terdeteksi, sedangkan eksperimen kedua menghasilkan 7 episode. Nilai tersebut menunjukkan frekuensi pencarian memasuki kondisi local maximum pada lintasan state yang diterima, bukan jumlah iterasi yang dihabiskan dalam kondisi tersebut. Dengan demikian, nilai stuck episodes tidak dapat digunakan secara langsung sebagai ukuran lamanya algoritma terjebak.

Dari sisi efisiensi, seluruh eksperimen diselesaikan dalam 1.000 iterasi dengan durasi pencarian utama antara 0,1066 hingga 0,1288 detik. Durasi relatif singkat karena setiap iterasi hanya membangkitkan satu neighbor acak. Secara keseluruhan, Simulated Annealing mampu meningkatkan nilai objective secara signifikan dibandingkan state awal, meskipun solusi terbaik yang diperoleh belum dapat dipastikan sebagai global optimum hanya berdasarkan tiga eksperimen tersebut.

2.3.6. Algoritma *Genetic Algorithm*

Pada seluruh eksperimen *Genetic Algorithm*, jumlah populasi dan banyak iterasi bervariasi, sementara parameter lainnya menggunakan nilai bawaan yang sama, yaitu:

Parameter	Nilai

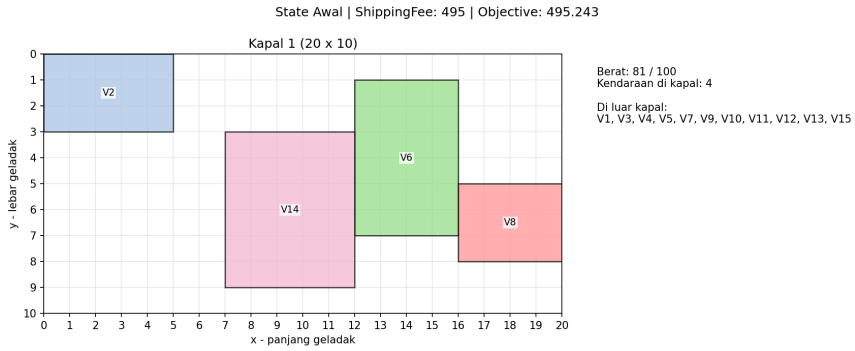
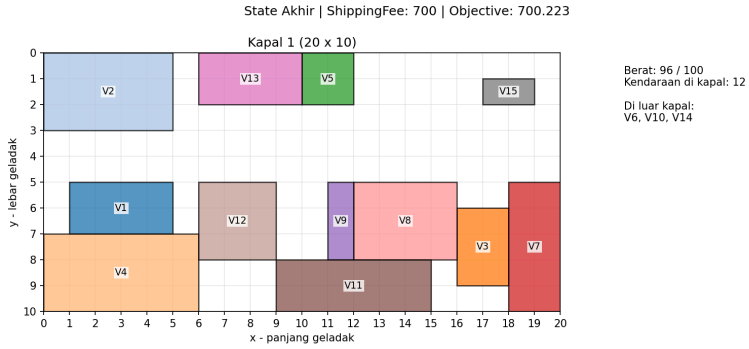
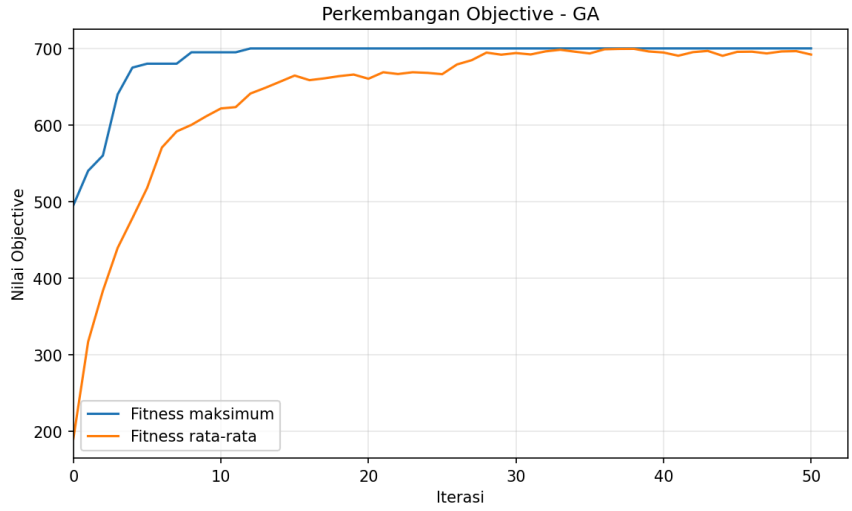
p (peluang kendaraan berada di kapal di awal)	0,3
crossover_rate (peluang terjadinya crossover)	0,9
mutation_rate (peluang terjadinya mutasi)	0,3
elite (banyak individu terbaik yang dipertahankan)	1

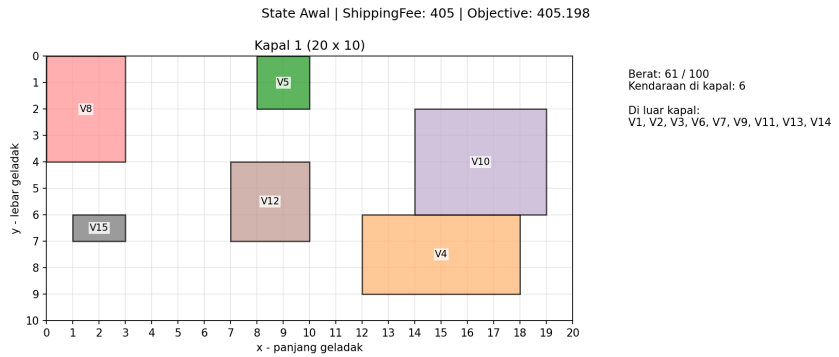
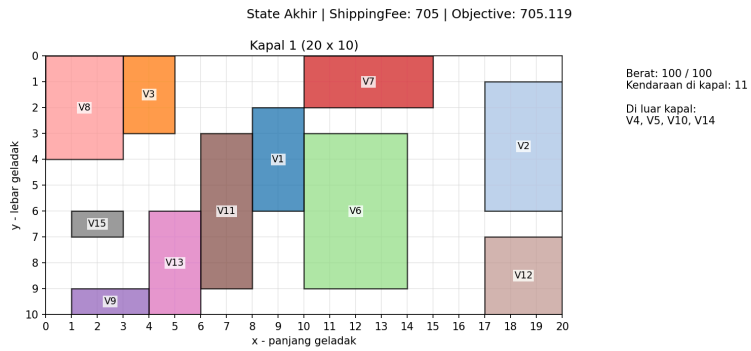
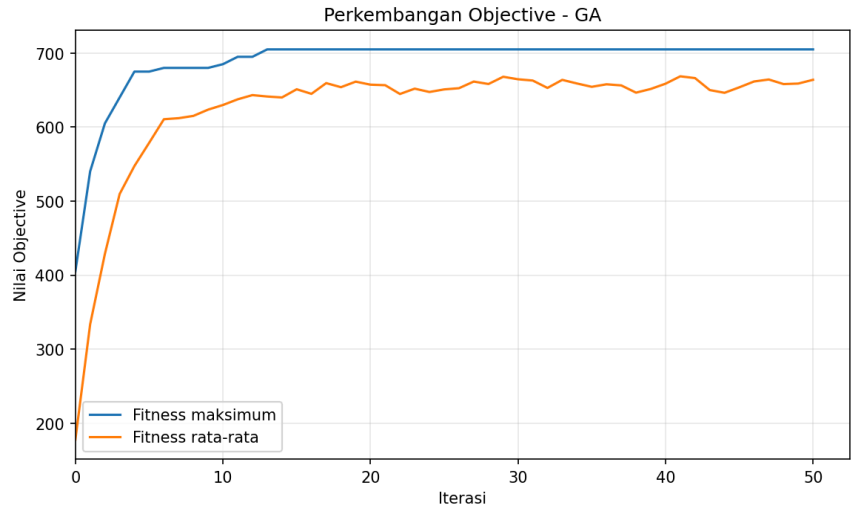
Nilai bawaan ini dipilih karena 1)Nilai p = 0,3 mengikuti nilai bawaan pembangkitan *state* awal pada program, sehingga setiap individu awal hanya memuat sebagian kendaraan. Dengan begitu, peluang individu awal untuk valid lebih besar, tetapi susunannya tetap beragam. 2)Nilai crossover_rate = 0,9 dibuat tinggi karena *crossover* merupakan operator utama untuk menggabungkan susunan yang baik dari dua induk, sementara pada 10% sisanya anak merupakan salinan induk secara utuh agar susunan yang sudah baik tidak selalu terpecah. 3)Nilai mutation_rate = 0,3 berlaku untuk setiap anak, karena mutasi pada program ini berupa satu *move* acak (menukar, menggeser, memutar, mengeluarkan, atau memasukkan kendaraan) terhadap seluruh susunan. Nilai ini cukup untuk menjaga keragaman populasi tanpa terlalu sering merusak hasil *crossover*. 4)Nilai elite = 1 menjamin individu terbaik selalu bertahan ke generasi berikutnya sehingga nilai *fitness* maksimum tidak pernah turun, tetapi hanya satu individu yang dipertahankan agar keragaman populasi tetap terjaga. Keempat nilai ini dibuat sama pada seluruh eksperimen sehingga perbedaan hasil hanya dipengaruhi oleh jumlah populasi dan banyak iterasi.

2.3.6.1. Populasi Sebagai Kontrol

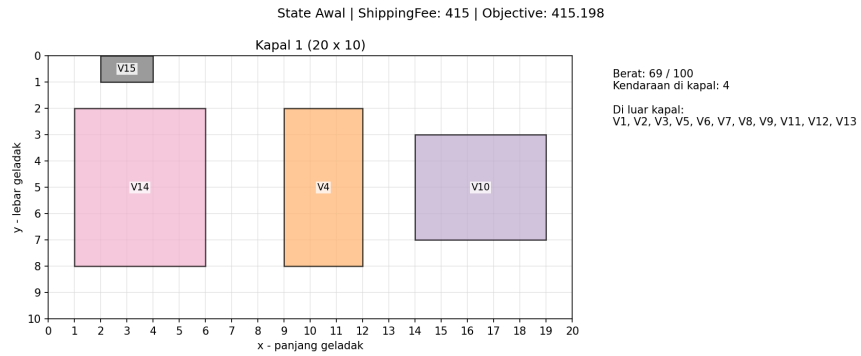
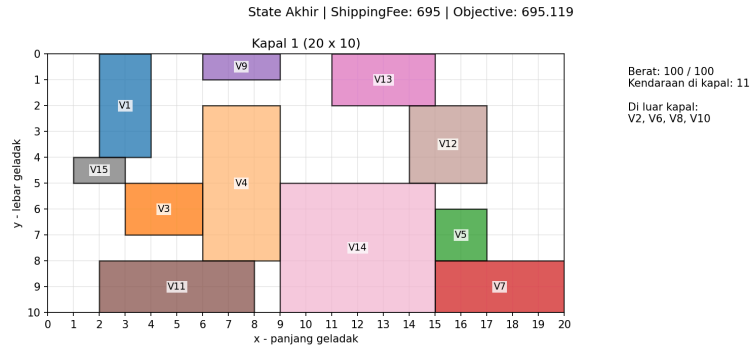
Berikut merupakan hasil eksperimen algoritma *Genetic Algorithm* dengan jumlah populasi sebagai kontrol sebesar 50 dan banyak iterasi 50, 100, dan 200 pada data sedang.json. Setiap konfigurasi dijalankan tiga kali dengan *seed* 1, 2, dan 3.

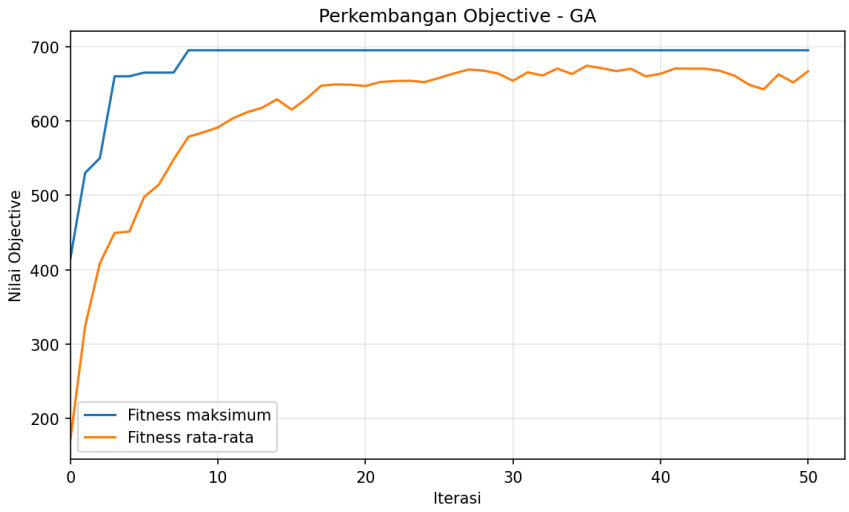
Eksperimen 1	
Jumlah Populasi	50
Banyak Iterasi	50

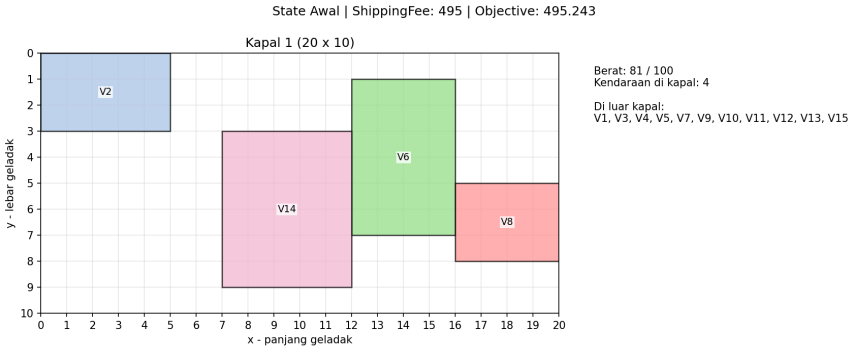
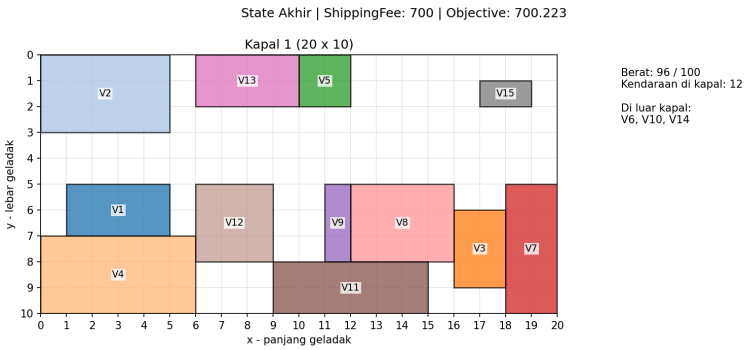
State Awal	
State Akhir	
Nilai *objective function* yang dicapai	700.223
Plot nilai *objective function* terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	0.526

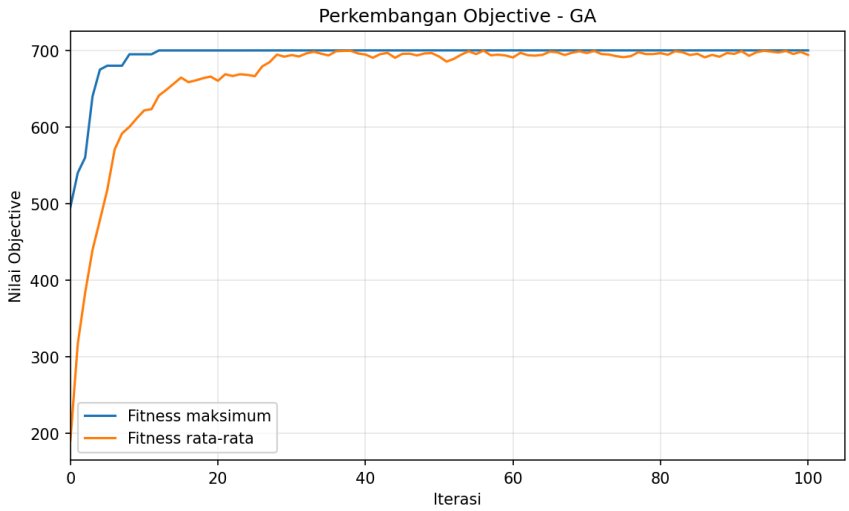
Eksperimen 2	
Jumlah Populasi	50
Banyak Iterasi	50
State Awal	
State Akhir	
Nilai *objective function* yang dicapai	705.119
Plot nilai *objective function* terhadap banyak iterasi yang telah dilewati	

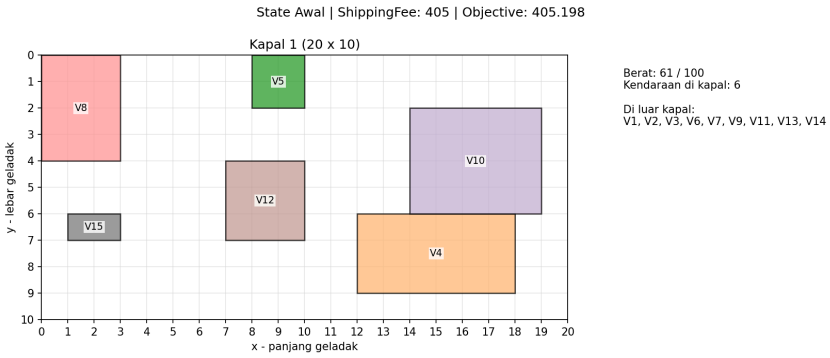
Durasi proses pencarian	0.526

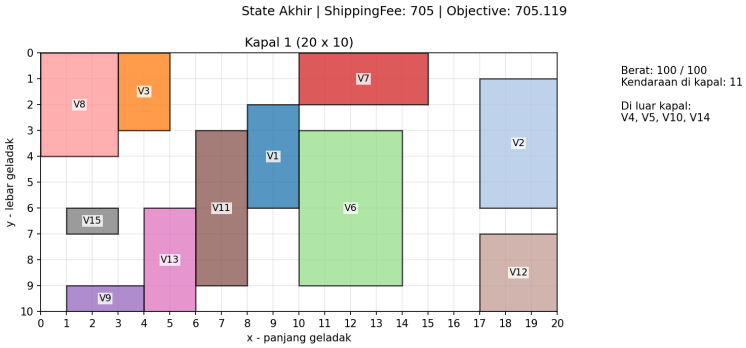
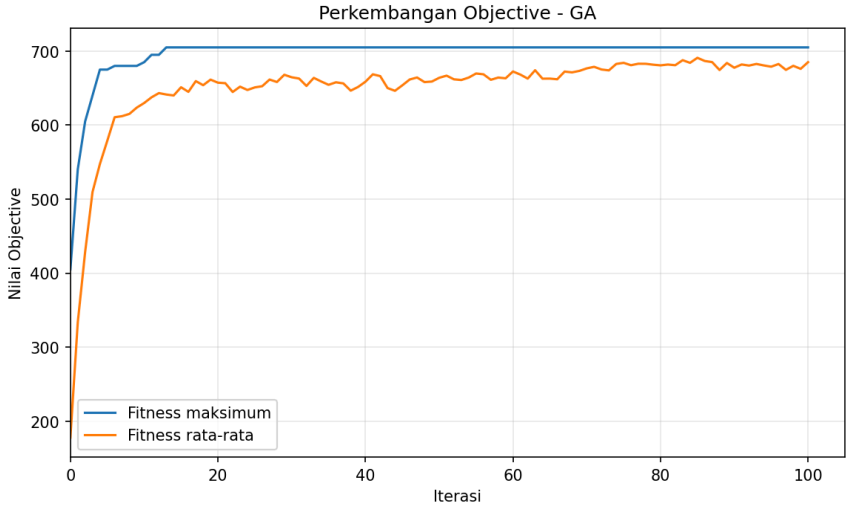
Eksperimen 3	
Jumlah Populasi	50
Banyak Iterasi	50
State Awal	
State Akhir	
Nilai *objective function* yang dicapai	695.119

Plot nilai *objective function* terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	0.471

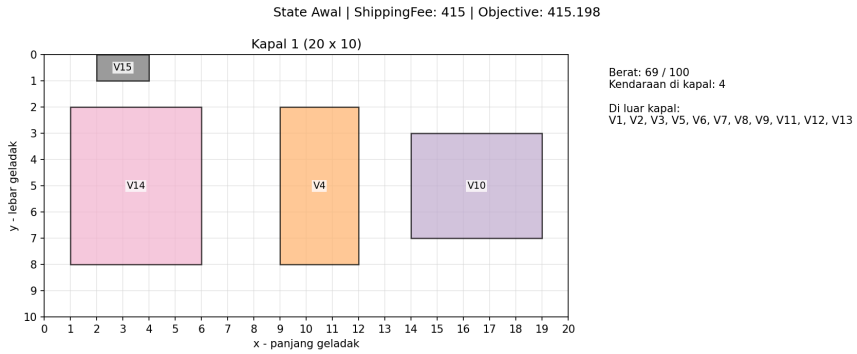
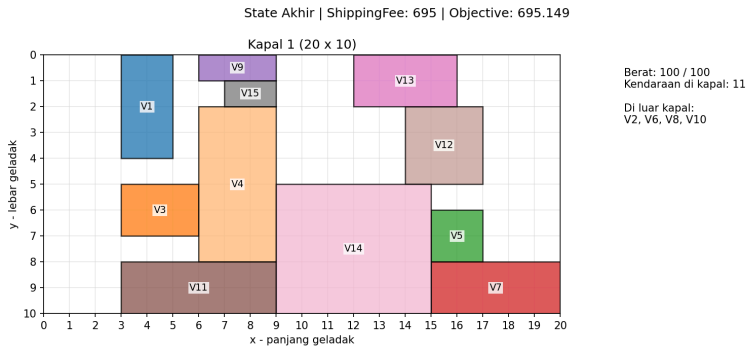
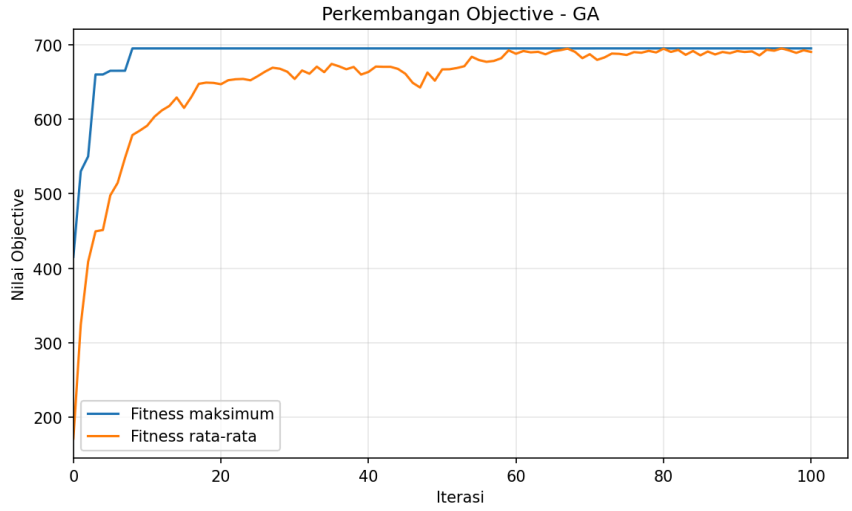
Eksperimen 4	
Jumlah Populasi	50
Banyak Iterasi	100
State Awal	
State Akhir	

Nilai *objective function* yang dicapai	700.223
Plot nilai *objective function* terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	1.036

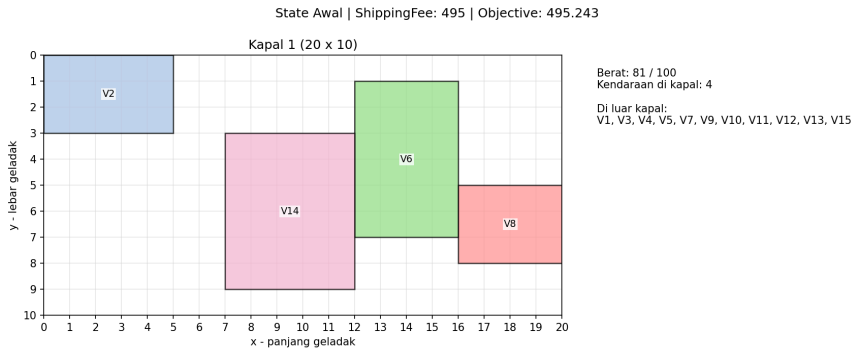
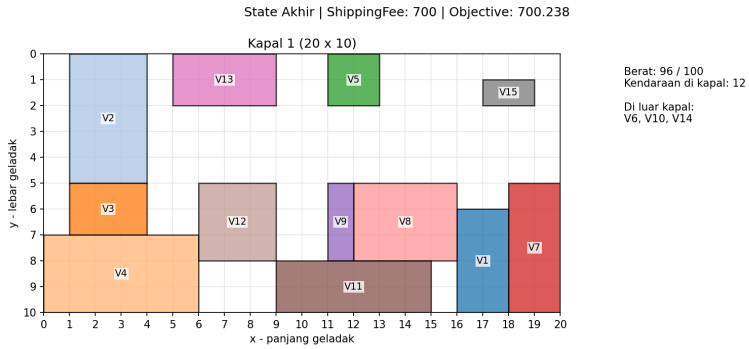
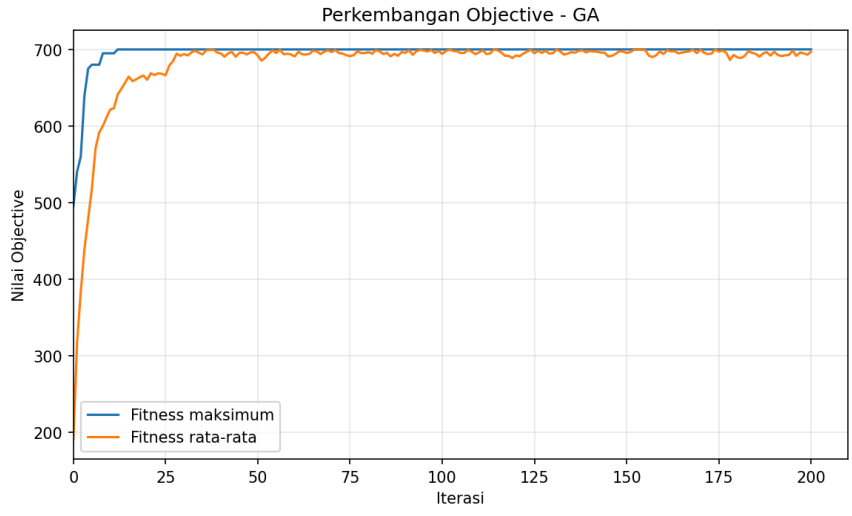
Eksperimen 5	
Jumlah Populasi	50
Banyak Iterasi	100
State Awal	

State Akhir	
Nilai *objective function* yang dicapai	705.119
Plot nilai *objective function* terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	1.066

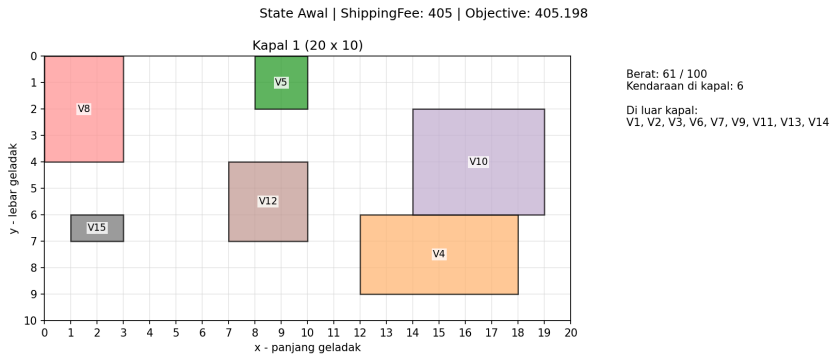
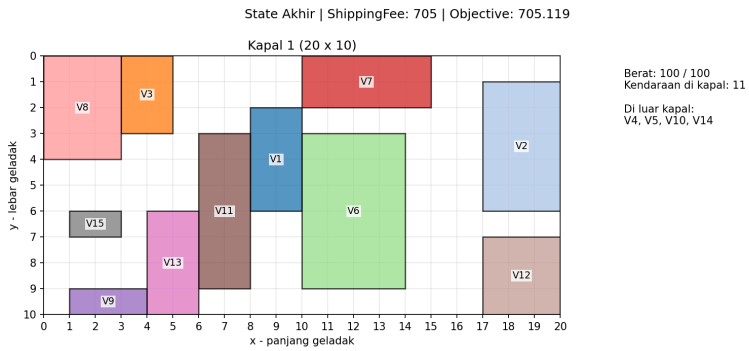
Eksperimen 6	
Jumlah Populasi	50
Banyak Iterasi	100

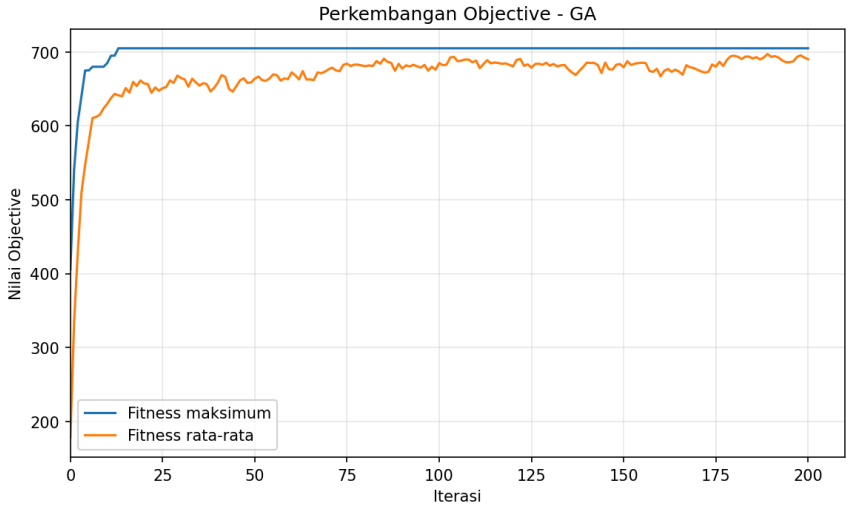
State Awal	
State Akhir	
Nilai *objective function* yang dicapai	695.149
Plot nilai *objective function* terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	0.924

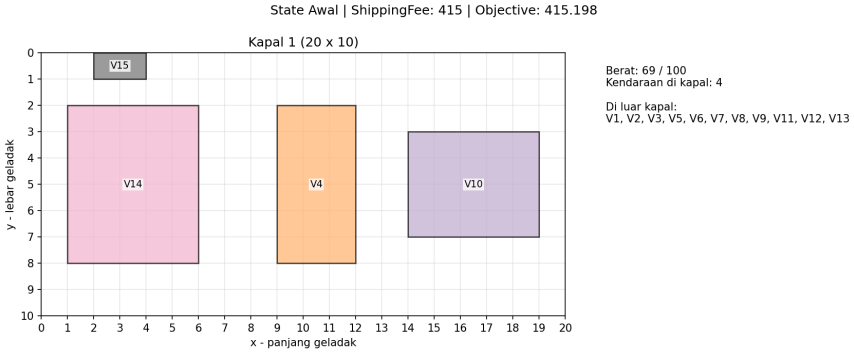
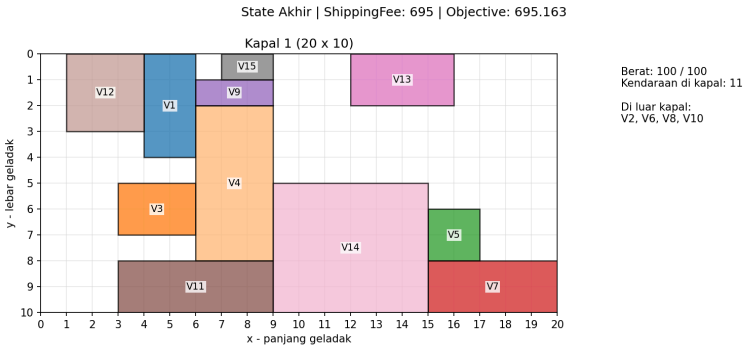
Eksperimen 7

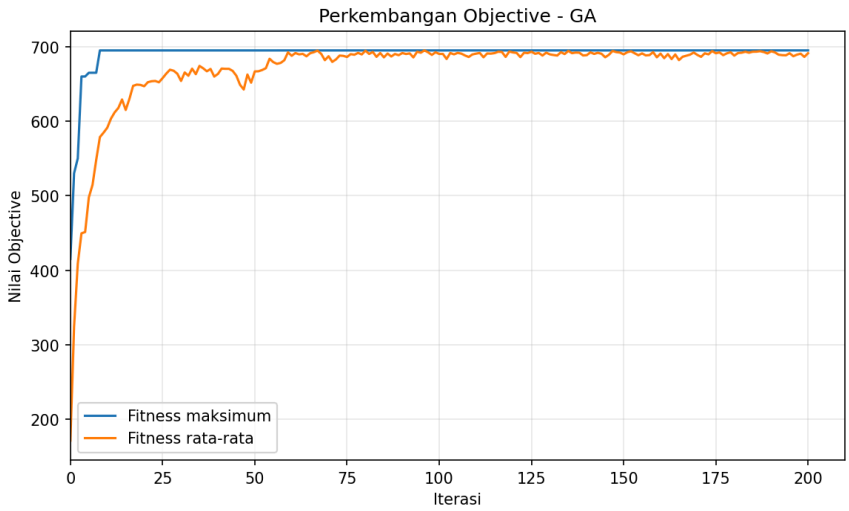
Jumlah Populasi	50
Banyak Iterasi	200
State Awal	
State Akhir	
Nilai *objective function* yang dicapai	700.238
Plot nilai *objective function* terhadap banyak iterasi yang telah dilewati	
Durasi proses	2.076

pencarian	

Eksperimen 8	
Jumlah Populasi	50
Banyak Iterasi	200
State Awal	
State Akhir	
Nilai *objective function* yang dicapai	705.119

Plot nilai *objective function* terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	2.080

Eksperimen 9	
Jumlah Populasi	50
Banyak Iterasi	200
State Awal	
State Akhir	

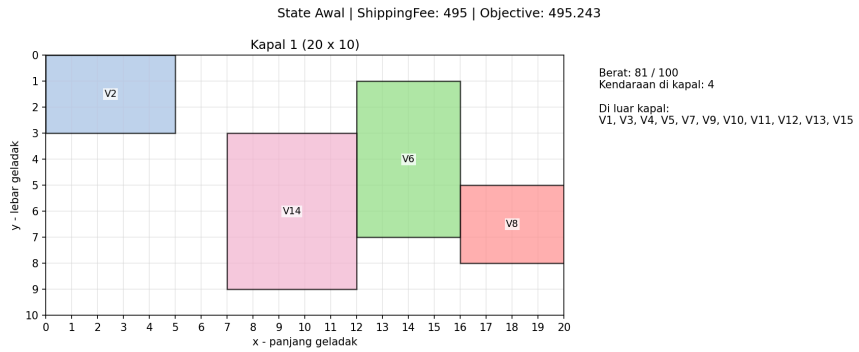
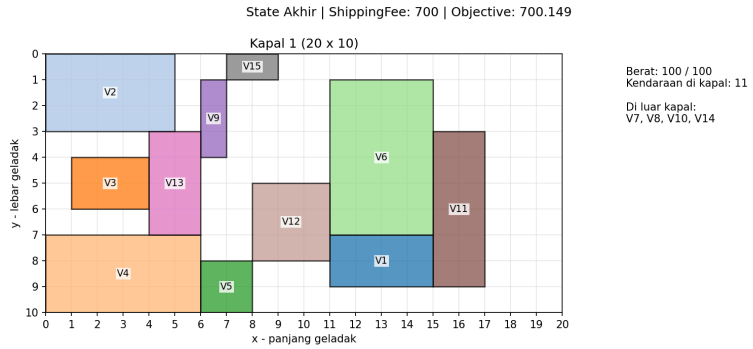
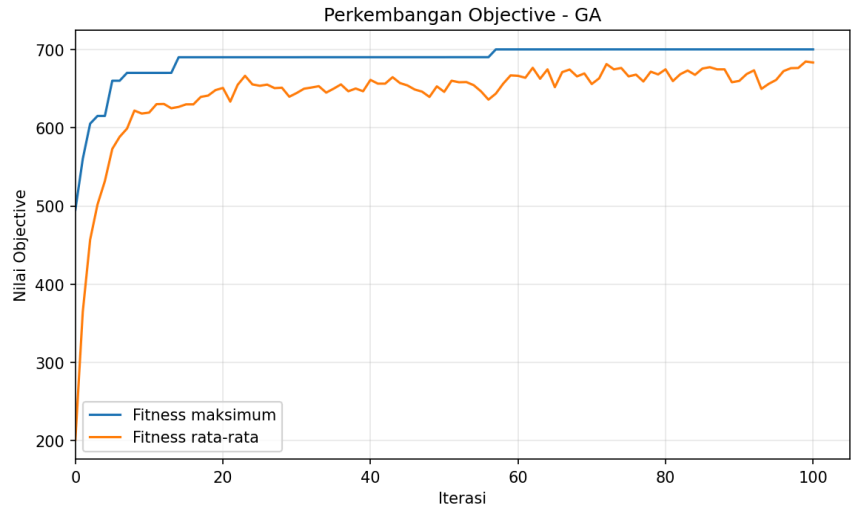
Nilai *objective function* yang dicapai	695.163
Plot nilai *objective function* terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	1.789

Berdasarkan sembilan eksperimen dengan jumlah populasi 50, penambahan banyak iterasi tidak mengubah total *ShippingFee* akhir. Pada *seed* yang sama, total *ShippingFee* tetap 700 (Eksperimen 1, 4, 7), 705 (Eksperimen 2, 5, 8), dan 695 (Eksperimen 3, 6, 9) pada iterasi 50, 100, maupun 200, karena nilai tersebut sudah tercapai pada iterasi ke-8 hingga ke-13. Setelah itu, populasi hanya menemukan susunan dengan total *ShippingFee* yang sama tetapi ruang kosong yang lebih luas, sehingga nilai *objective* naik sedikit melalui *tie-breaker*, misalnya dari 695,119 menjadi 695,149 dan 695,163. Sebaliknya, durasi pencarian naik hampir sebanding dengan banyak iterasi, yaitu sekitar 0,5 detik, 1 detik, dan 2 detik. Hal ini menunjukkan bahwa dengan populasi 50, *Genetic Algorithm* cepat konvergen, sehingga penambahan iterasi lebih banyak menambah waktu daripada memperbaiki hasil. Hasil akhir justru lebih ditentukan oleh populasi awal, *seed* 2 mencapai 705, sedangkan *seed* 3 tertahan di 695.

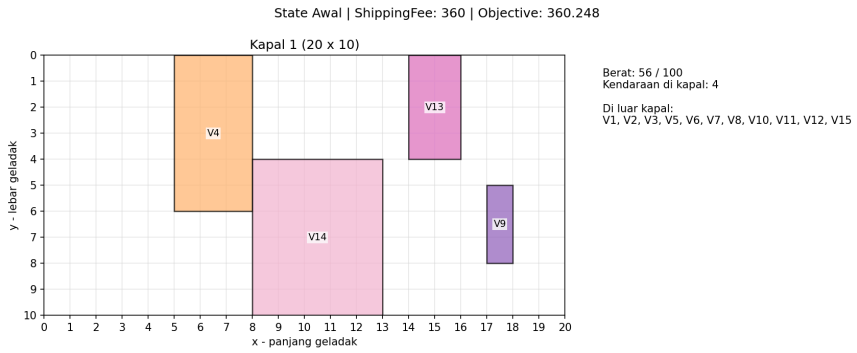
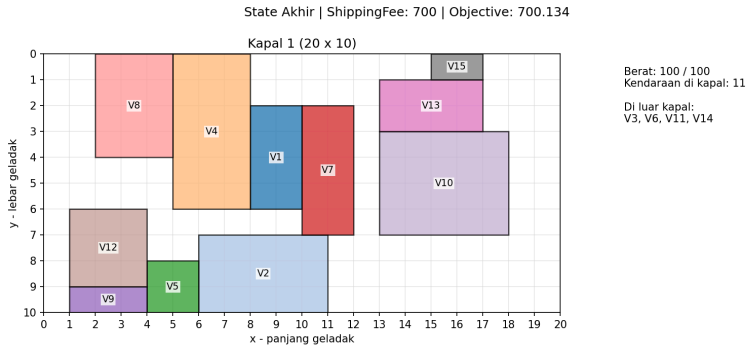
2.3.6.2. Iterasi Sebagai Kontrol

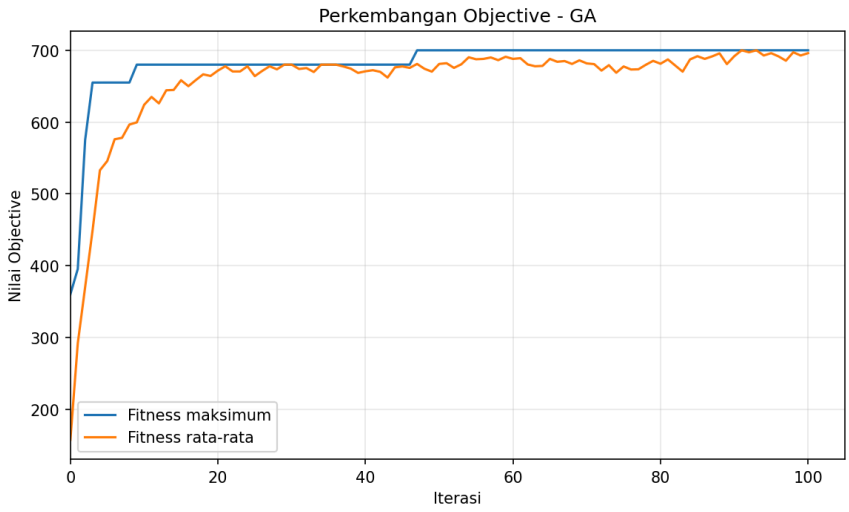
Berikut merupakan hasil eksperimen algoritma *Genetic Algorithm* dengan banyak iterasi sebagai kontrol sebesar 100 dan jumlah populasi 20, 100, dan 200 pada data sedang.json. Setiap konfigurasi dijalankan tiga kali dengan *seed* 1, 2, dan 3.

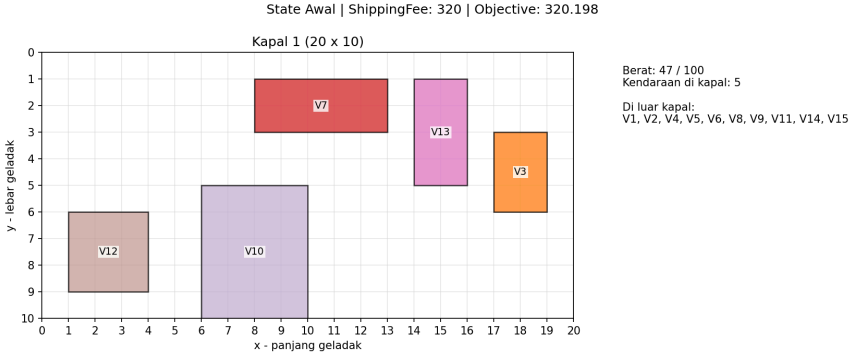
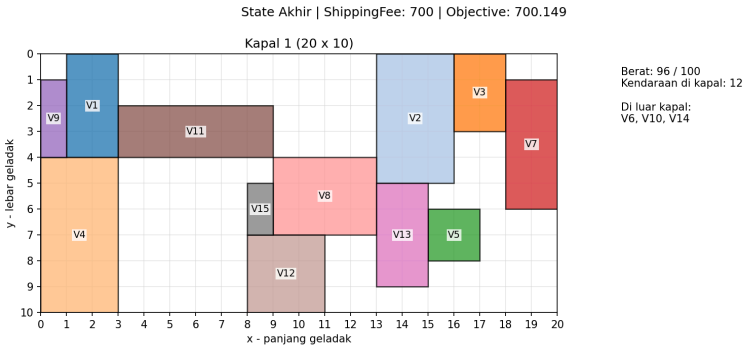
Eksperimen 1

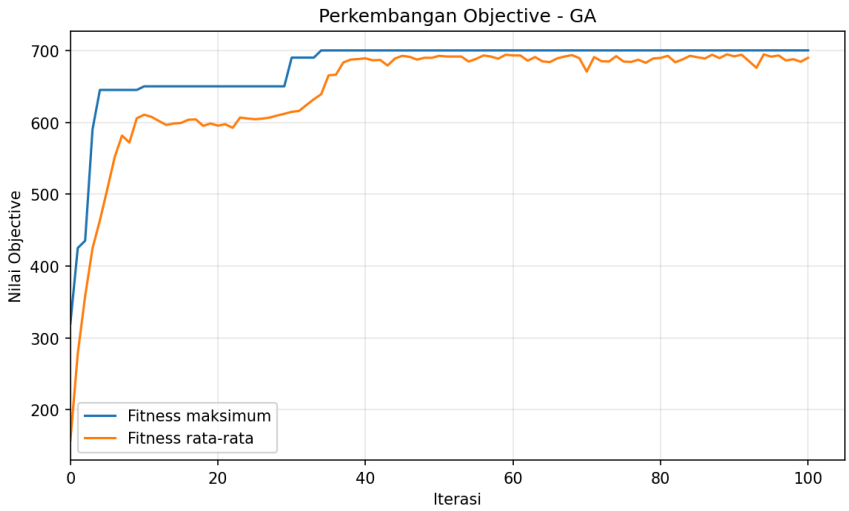
Jumlah Populasi	20
Banyak Iterasi	100
State Awal	
State Akhir	
Nilai *objective function* yang dicapai	700.149
Plot nilai *objective function* terhadap banyak iterasi yang telah dilewati	
Durasi proses	0.402

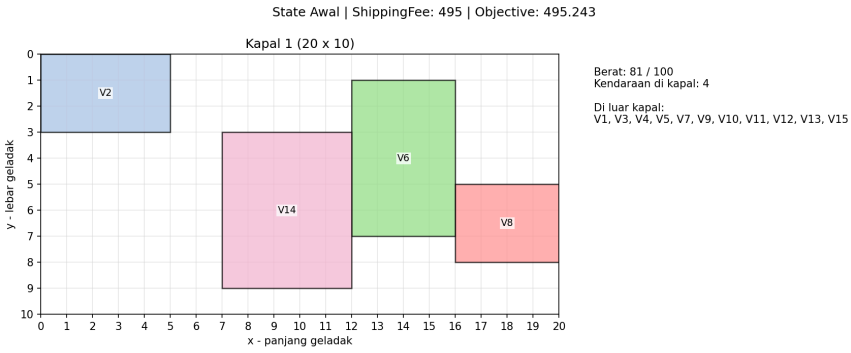
pencarian	

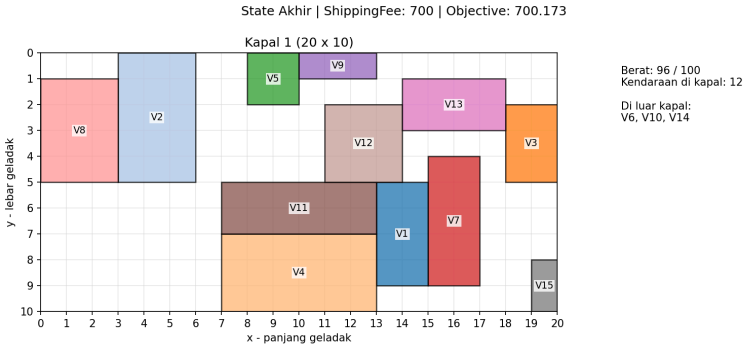
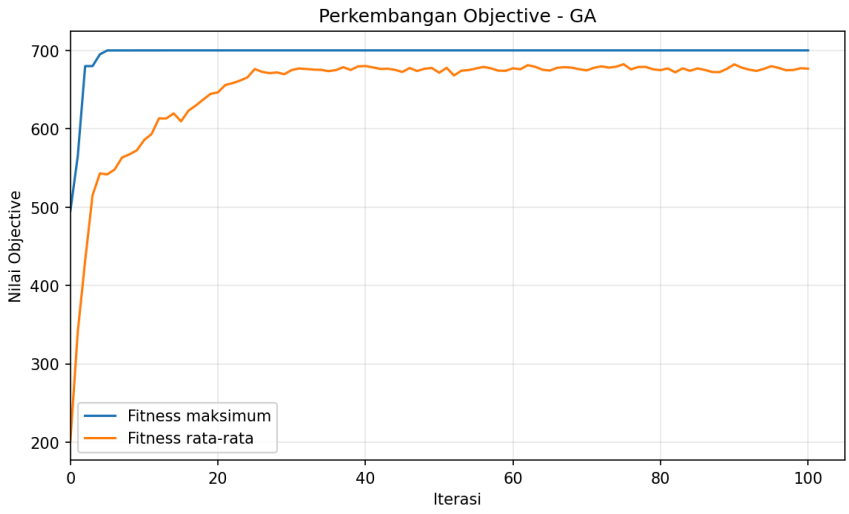
Eksperimen 2	
Jumlah Populasi	20
Banyak Iterasi	100
State Awal	
State Akhir	
Nilai *objective function* yang dicapai	700.134

Plot nilai *objective function* terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	0.372

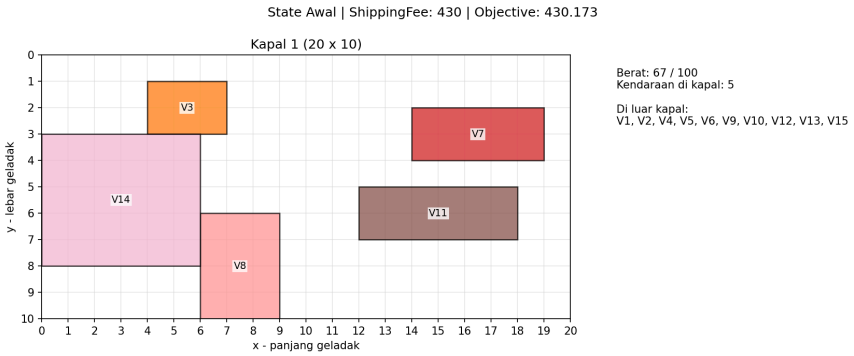
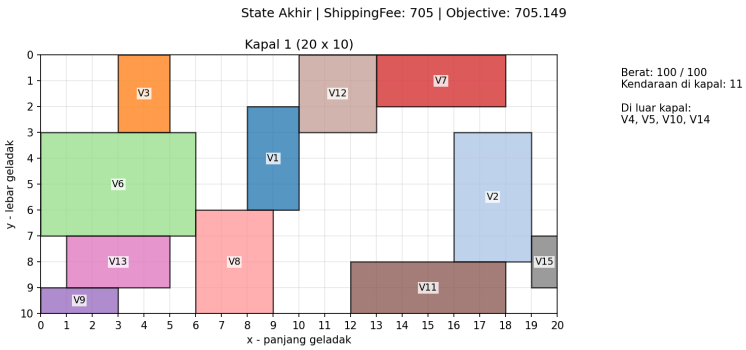
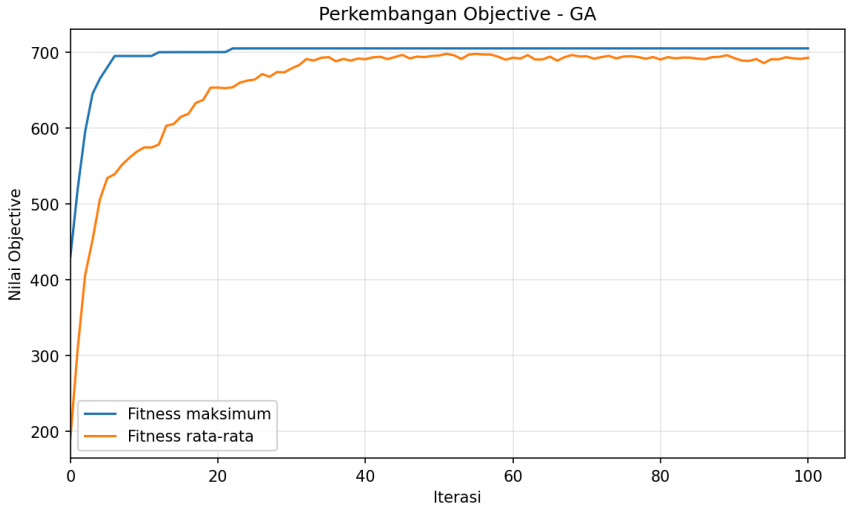
Eksperimen 3	
Jumlah Populasi	20
Banyak Iterasi	100
State Awal	
State Akhir	

Nilai *objective function* yang dicapai	700.149
Plot nilai *objective function* terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	0.383

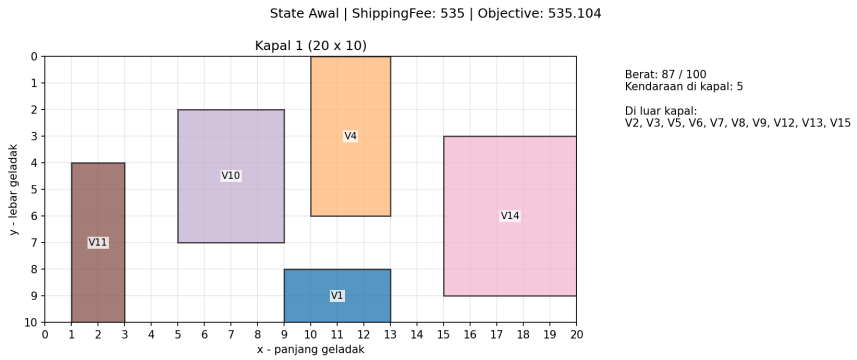
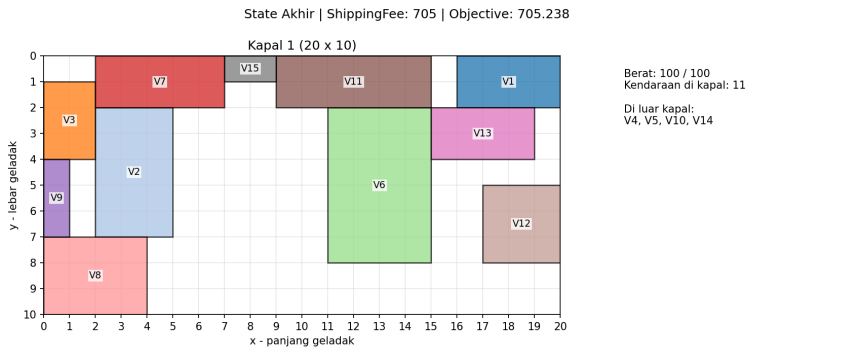
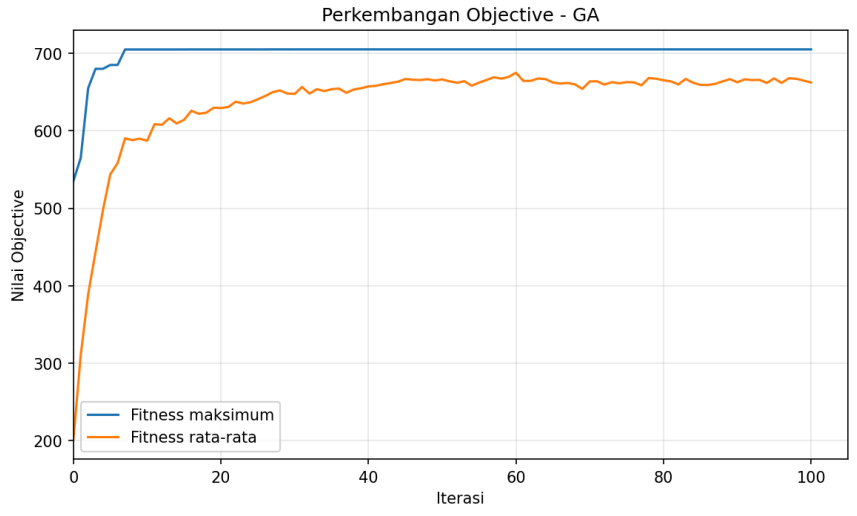
Eksperimen 4	
Jumlah Populasi	100
Banyak Iterasi	100
State Awal	

State Akhir	
Nilai *objective function* yang dicapai	700.173
Plot nilai *objective function* terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	2.245

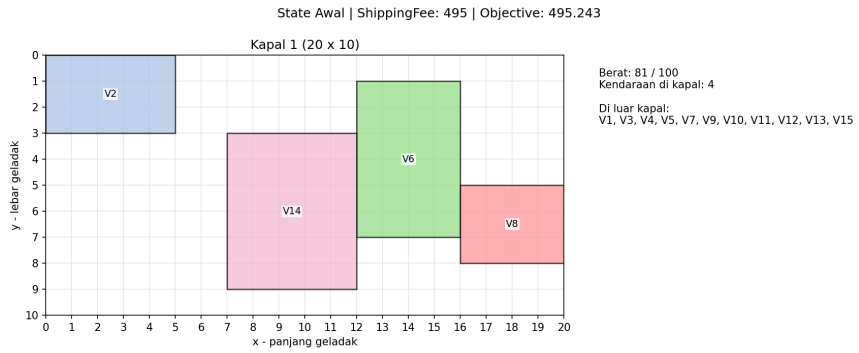
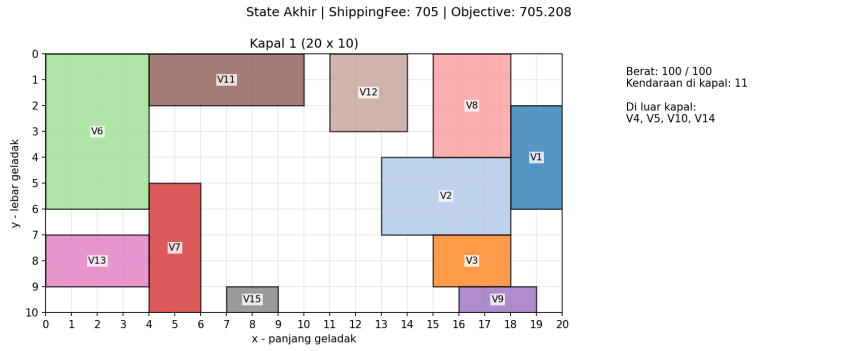
Eksperimen 5	
Jumlah Populasi	100
Banyak Iterasi	100

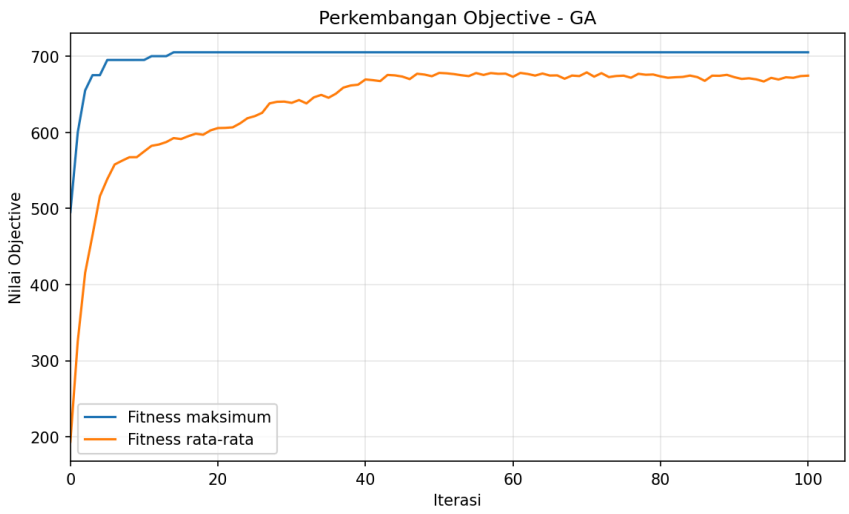
State Awal	
State Akhir	
Nilai *objective function* yang dicapai	705.149
Plot nilai *objective function* terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	2.017

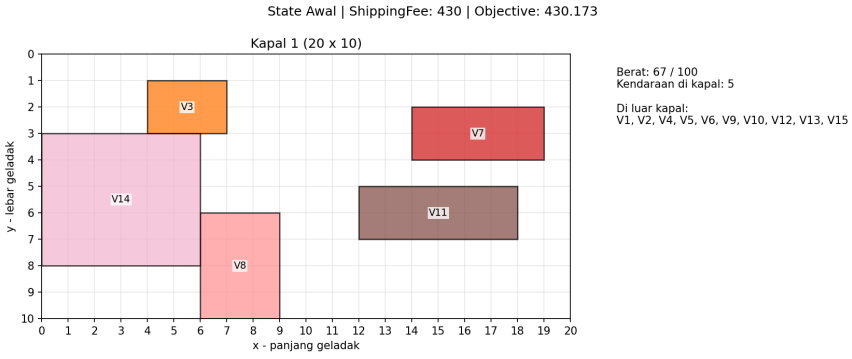
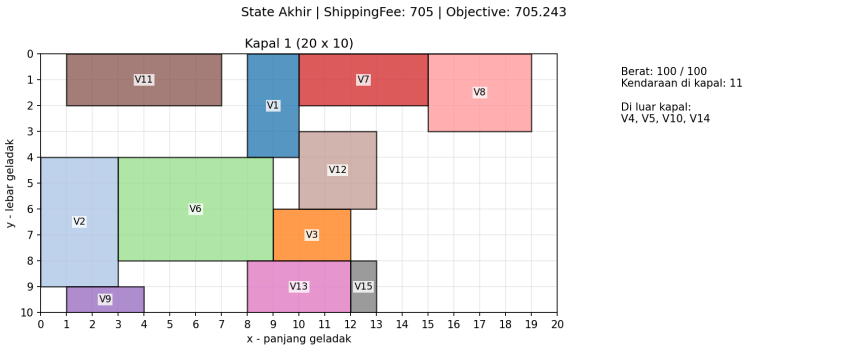
Eksperimen 6

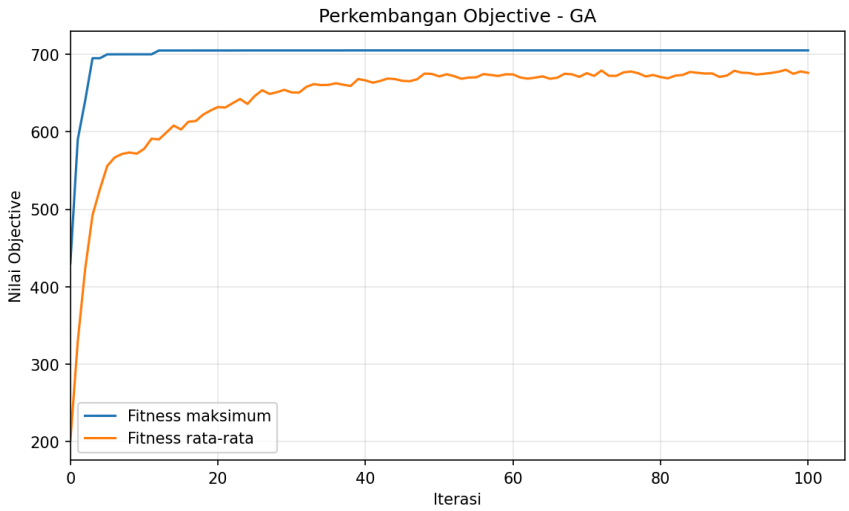
Jumlah Populasi	100
Banyak Iterasi	100
State Awal	
State Akhir	
Nilai *objective function* yang dicapai	705.238
Plot nilai *objective function* terhadap banyak iterasi yang telah dilewati	
Durasi proses	2.267

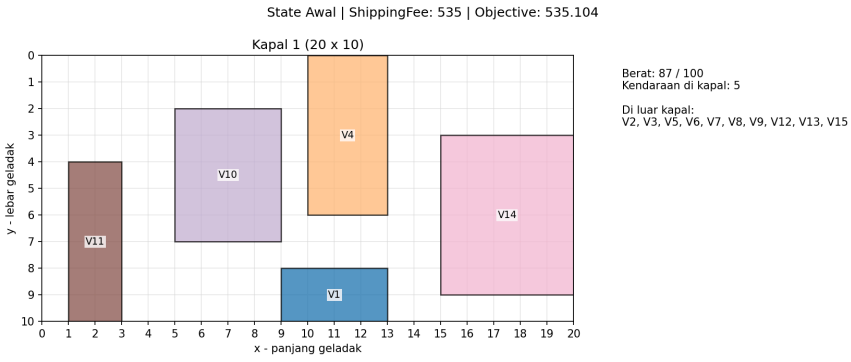
pencarian	

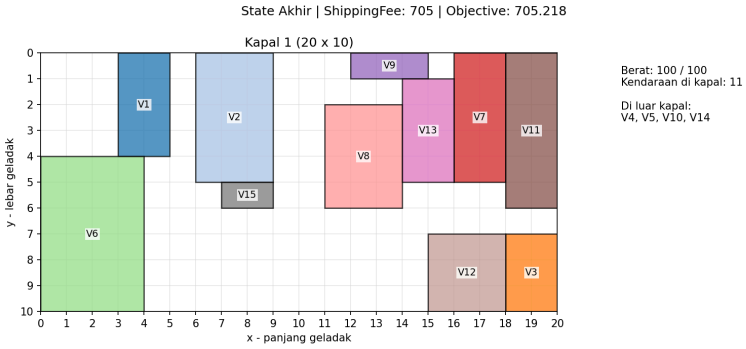
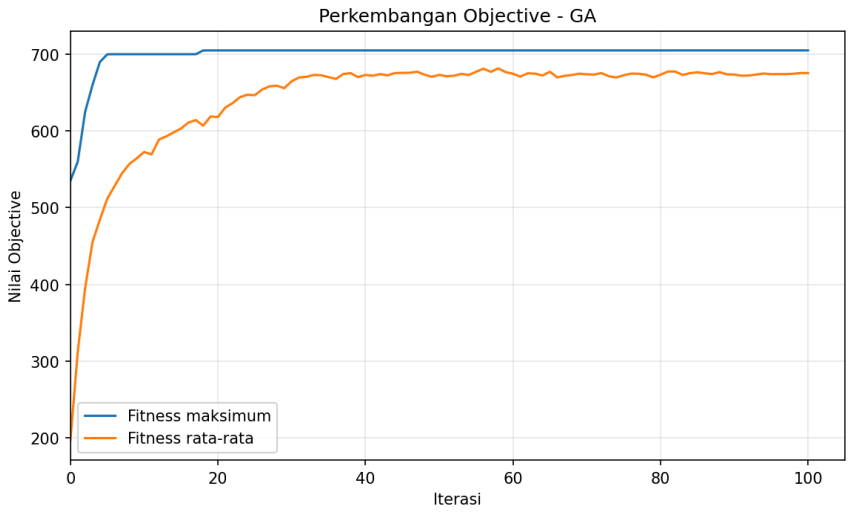
Eksperimen 7	
Jumlah Populasi	200
Banyak Iterasi	100
State Awal	
State Akhir	
Nilai *objective function* yang dicapai	705.208

Plot nilai *objective function* terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	4.533

Eksperimen 8	
Jumlah Populasi	200
Banyak Iterasi	100
State Awal	
State Akhir	

Nilai *objective function* yang dicapai	705.243
Plot nilai *objective function* terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	4.666

Eksperimen 9	
Jumlah Populasi	200
Banyak Iterasi	100
State Awal	

State Akhir	
Nilai *objective function* yang dicapai	705.218
Plot nilai *objective function* terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	4.787

Berbeda dengan banyak iterasi, jumlah populasi berpengaruh nyata terhadap hasil. Dengan populasi 20, ketiga eksperimen berakhir pada total *ShippingFee* 700 dan membutuhkan 34 hingga 57 iterasi untuk mencapainya. Dengan populasi 100, dua dari tiga eksperimen mencapai 705, sedangkan dengan populasi 200 ketiga eksperimen mencapai 705 dalam 12 hingga 18 iterasi. Nilai 705 merupakan total *ShippingFee* terbesar yang mungkin dicapai pada data ini, karena kombinasi kendaraan terbaik yang memenuhi batas berat 100 dan luas geladak juga bernilai 705, sehingga hasil tersebut merupakan *global optimum*. Populasi yang lebih besar menyediakan lebih banyak variasi susunan sejak awal, sehingga peluang terbentuknya kombinasi kendaraan terbaik melalui *crossover* lebih besar dan hasilnya lebih konsisten. Konsekuensinya, durasi pencarian naik

hampir sebanding dengan jumlah populasi, yaitu sekitar 0,4 detik, 2,2 detik, dan 4,7 detik, karena setiap iterasi harus membentuk dan mengevaluasi lebih banyak anak.

3. Kesimpulan dan Saran

Kesimpulan :

1. Algoritma local search berhasil diterapkan untuk mencari solusi penataan kendaraan pada kapal kargo dengan memaksimalkan total ShippingFee, serta tetap memenuhi batasan ukuran geladak, posisi kendaraan, dan kapasitas berat maksimum.
2. Genetic Algorithm menghasilkan solusi terbaik di antara algoritma yang diuji, dengan ShippingFee tertinggi sebesar 705. Peningkatan jumlah populasi juga menghasilkan solusi yang lebih konsisten, tetapi membutuhkan waktu komputasi lebih lama.
3. Stochastic Hill-Climbing merupakan algoritma dengan waktu pencarian tercepat, yaitu sekitar 0,034–0,040 detik, serta menghasilkan rata-rata ShippingFee tertinggi di antara varian Hill-Climbing. Sementara itu, Steepest Ascent cenderung cepat mencapai local maximum dan sangat bergantung pada state awal.
4. Simulated Annealing menghasilkan ShippingFee terbaik sebesar 650, 635, dan 675 pada tiga eksperimen. Mekanisme penerimaan neighbor yang lebih buruk memungkinkan eksplorasi solusi, tetapi tidak selalu menghasilkan solusi yang lebih baik dibandingkan algoritma lainnya.
5. Secara keseluruhan, tidak ada algoritma yang unggul pada seluruh aspek. Genetic Algorithm lebih unggul dalam kualitas solusi, sedangkan Stochastic Hill-Climbing lebih efisien dalam waktu pencarian. Pemilihan algoritma perlu mempertimbangkan keseimbangan antara kualitas hasil dan biaya komputasi.

Saran :

Pengembangan selanjutnya dapat dilakukan dengan menguji dataset yang lebih besar dan beragam serta menambah jumlah percobaan untuk menilai konsistensi hasil. Selain itu, optimasi parameter pada Simulated Annealing dan Genetic Algorithm serta perbaikan mekanisme pembangkitan neighbor dapat dilakukan untuk meningkatkan kualitas solusi dan efisiensi pencarian.

:

4. Pembagian Tugas

NIM	Nama Anggota	Pembagian Tugas
18224050	Syafaq Jiwa Raharjo	• Mengerjakan implementasi *Genetic Algorithm* • Menambahkan metode input jumlah populasi dan banyak iterasi *Genetic Algorithm* pada menu program utama • Mengerjakan implementasi, hasil eksperimen, dan analisis *Genetic Algorithm* pada Laporan Tugas Besar
18224086	Faiz Muhammad Sauqi	• Mengerjakan fondasi program (representasi *state*, *constraint*, *objective function*, *neighbor*, dan *initial state*) • Mengerjakan implementasi *hill-climbing* (*steepest ascent*, *sideways move*, *stochastic*, dan *random restart*) • Mengerjakan program utama, integrasi algoritma, dan README • Mengerjakan deskripsi persoalan, *objective function*, serta hasil eksperimen dan analisis untuk *hill-climbing* pada Laporan Tugas Besar
18224096	Azfar Arafi Shofyan	• Mengerjakan implementasi algoritma Simulated Annealing, termasuk cooling schedule, mekanisme penerimaan probabilistik, dan diagnosis local maximum.

		• Mengembangkan fungsi visualisasi state kendaraan, perkembangan objective function, serta grafik eksponensial untuk analisis Simulated Annealing. • Melakukan pengujian dan eksperimen Simulated Annealing menggunakan dataset kecil dan sedang dengan beberapa random seed. • Mengerjakan bagian implementasi, hasil eksperimen, dan analisis Simulated Annealing pada Laporan Tugas Besar.

5. Referensi

GeeksforGeeks. (2023, August 16). *Introduction to hill climbing | Artificial intelligence*. GeeksforGeeks. https://www.geeksforgeeks.org/artificial-intelligence/introduction-hill-climbing-artificial-intelligence/

Tim Pengajar IF3070. (2026). *Beyond classical search* [Slide kuliah, IF3070 Dasar Inteligensi Artifisial]. Institut Teknologi Bandung.

GeeksforGeeks. (2024, April 8). *Simulated Annealing*. https://www.geeksforgeeks.org/dsa/simulated-annealing/

GeeksforGeeks. (2024, June 20). Introduction of Genetic Algorithm. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/genetic-algorithms/

TutorialsPoint. (n.d.). Genetic Algorithms - Introduction. TutorialsPoint. https://www.tutorialspoint.com/genetic_algorithms/genetic_algorithms_introduction.htm

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, ~~saya~~/kami*:

Nama/NIM : Syafaq J.R./18224050, F.M. Sauqi/18224086, Azfar A.S./ 18224096

Fakultas/Sekolah : STEI-K

Kode Mata Kuliah : IF-3070

Nama Mata Kuliah : Dasar Intelegensi Artifisial

Judul Tugas/Proposal :PENCARIAN SOLUSI PENATAAN KENDARAAN PADA KAPAL KARGO DENGAN LOCAL SEARCH

Menyatakan bahwa ~~saya~~/kami* menggunakan/~~tidak~~ menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	**Digunakan ?**	**Tingkat Penggunaan**
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	~~Ya~~/Tidak*	*No AI*
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya/~~Tidak~~*	*AI-assisted idea generation and structuring*
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya/~~Tidak~~*	*AI-assisted workflow*
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	~~Ya~~/Tidak*	*No AI*
Lainnya, Jelaskan:	**-**	**-**

~~Saya~~/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026



Syafaq Jiwa Raharjo

18224050